

Grado Universitario en Ingeniería Informática
2024-2025

Trabajo Fin de Grado

“Revisión de estrategias de búsqueda en espacios con incertidumbre”

Yago Brotón Gutiérrez

Tutores

Juan Pedro Llerena Caña

Jesus García Herrero

Leganés, Febrero de 2025



Esta obra se encuentra sujeta a la licencia Creative Commons **Reconocimiento - No Comercial - Sin Obra Derivada**

RESUMEN

La búsqueda de objetivos en un espacio donde su ubicación no se conoce con certeza, pero se dispone de ciertos indicios provenientes de una o varias fuentes de información, se denomina búsqueda en espacios con incertidumbre. Los principales enfoques en este campo se enmarcan dentro de la teoría de búsqueda bayesiana, cuyos orígenes se remontan al *Anti-Submarine Warfare Operations Research Group* (ASWORG) durante la Segunda Guerra Mundial, que tenía como objetivo proporcionar herramientas para interceptar submarinos enemigos.

Estos enfoques abordan de manera explícita la incertidumbre inherente a las distintas fuentes de información: los indicios disponibles para la búsqueda, la incertidumbre asociada al comportamiento esperado de los objetivos, la dinámica de los vehículos o agentes encargados de la búsqueda, así como la incertidumbre derivada de los sensores utilizados para la detección. Generalmente, el objetivo principal de estas metodologías es maximizar la probabilidad de identificar los objetivos y/o minimizar el tiempo necesario para su localización.

Aunque existen numerosos trabajos en este ámbito, la mayoría se centran en destacar la contribución científica presentada, sin proporcionar herramientas software que permitan replicar los resultados y avanzar en el estado del arte. En este Trabajo de Fin de Grado se desarrolla un software basado en un enfoque discreto del problema de búsqueda en espacios con incertidumbre, empleando uno y varios agentes que simulan el comportamiento de vehículos aéreos no tripulados y se propone una metodología para la comparación de estrategias mediante un conjunto de métricas de calidad.

Palabras clave: Búsqueda en espacios con incertidumbre, Búsqueda Bayesiana, UAVs

DEDICATORIA

Quiero agradecer a todos los que me han ayudado y acompañado en la realización de este trabajo, mis padres y mis hermanos que me apoyaron a pesar de no entender completamente de qué iba esta tesis, pero aun así hicieron todo lo posible por ayudarme y a mis tutores Jompy y Jesús por su orientación, sin la que este proyecto no habría salido adelante.

ÍNDICE GENERAL

1. INTRODUCCIÓN.	1
1.1. Motivación	1
1.2. Objetivos	1
1.3. Entorno socio-económico	1
1.4. Marco regulador	3
1.5. Estructura	4
2. ESTADO DEL ARTE.	5
3. MODELADO DEL PROBLEMA	6
3.1. Generación del espacio de búsqueda	6
3.2. Modelado del objetivo	7
3.3. Observaciones	9
3.4. Fusión de información: Filtro Recursivo Bayesiano	10
3.5. Ejemplo ilustrativo del RBF.	11
3.6. Notación	11
4. PROPUESTA DE TRABAJO	13
4.1. Problema de la miopía	13
4.2. Coordinación de varios agentes	14
4.2.1. Evitar colisiones	15
4.3. Requisitos.	15
4.3.1. Plantilla de requisitos	15
4.3.2. Requisitos funcionales.	16
4.3.3. Requisitos no funcionales	23
5. DESARROLLO DEL FRAMEWORK	26
5.1. Tecnologías y Frameworks	26
5.1.1. Librerías.	27
5.2. Generación de los casos de pruebas	28
5.2.1. Parámetros de los casos de prueba	28

5.3. Algoritmos de búsqueda	30
5.3.1. Búsqueda de barrido de cortacesped	31
5.3.2. Búsqueda de barrido en espiral	32
5.3.3. Búsqueda voraz.	33
5.4. Resultados de las pruebas	33
5.5. Interfaz gráfica	34
6. PROCESO DE EXPERIMENTACIÓN	35
6.1. Diseño de las pruebas	35
6.2. Tipos de pruebas	35
6.3. Ejecución de las pruebas.	37
6.3.1. Entorno de ejecución de las pruebas	38
7. ANÁLISIS DE RESULTADOS	39
7.1. Experimento 1: Diferencia entre algoritmos de búsqueda por fuerza bruta y algoritmos guiados por heurísticas	39
7.1.1. Caso 1	40
7.1.2. Caso 2: Posición inicial del agente fija	42
7.1.3. Caso 3: Ampliar el tamaño del área de búsqueda	49
7.1.4. Caso 4: Ampliar el tamaño del área de búsqueda manteniendo la posición del agente fija	52
7.2. Experimento 2: Búsqueda multi-indicio	53
7.3. Experimento 3: Búsqueda multiagente	55
8. CONCLUSIONES	60
8.1. Trabajo futuro	60
BIBLIOGRAFÍA	62

ÍNDICE DE FIGURAS

3.1	Ejemplos de espacios de búsqueda posibles a generar	7
3.2	Ejemplo ilustrativo del Filtro Recursivo Bayesiano	11
4.1	Ejemplos ilustrativos de la miopía, a partir de la tesis de P. Lanillos et al. [14]	13
4.2	Comparación del funcionamiento entre agente miope y un agente con la miopía reducida	14
5.1	Módulos principales del framework	28
5.2	Patrón de búsqueda de cortacesped	32
5.3	Patrón de búsqueda de cuadrado creciente	32
5.4	Patrón de búsqueda de voraz	33
5.5	Información obtenida en una búsqueda	34
6.1	Tipos de escenarios para comparar rendimiento entre algoritmos con un solo indicio.	36
6.2	Tipos de escenarios para comparar rendimiento entre algoritmos, con va- rios indicios.	37
6.3	Evolución del enjambre en la búsqueda multi-agente.	37
6.4	Diagrama de la ejecución de un caso de prueba.	38
7.1	Combinación de las situaciones iniciales del caso de prueba 1	40
7.2	Porcentajes de intentos Exitosos para cada Estrategia	41
7.3	Distribución de pasos y distancias de los casos exitosos	42
7.4	Combinación de las condiciones iniciales de las pruebas: (a) prueba con indicios fijos, (b) prueba con indicios en posiciones aleatorias	43
7.5	Porcentaje de intentos exitosos por estrategia	44
7.6	Distribución de los pasos tomados para cada estrategia - Posiciones fijas .	45
7.7	Porcentaje de intentos exitosos para cada estrategia - Posición del indicio aleatoria	46
7.8	Función de distribución de los intentos para cada estrategia - Posición del indicio aleatoria.	47

7.9	Distribución normal ajustada, superpuesta sobre la distribución de pasos para el planificador con la heurística de reducción de miopía.	48
7.10	Distribución de los pasos tomados por cada estrategia - Posición del indicio aleatoria.	49
7.11	Distribución de los pasos tomados por cada estrategia - Escenario grande.	50
7.12	Combinación de las situaciones iniciales del - Caso 3: Escenario grande. .	51
7.13	Porcentaje de intentos exitosos por estrategia - Caso 3: escenario grande. .	52
7.14	Distancia media recorrida por cada estrategia - Caso 4	53
7.15	Porcentaje de pruebas exitosas por estrategia - Experimento 2: Búsqueda multindicio	54
7.16	Agente guiado por la heurística global, rebotando entre los indicios	55
7.17	Progresión del tamaño del enjambre para el escenario de búsqueda mediano: (a) Diez agentes, (b) Cincuenta agentes, (c) Cien agentes	57
7.18	Desplazamiento y distancia recorrida por tamaño de enjambre - Escenario mediano	59

ÍNDICE DE TABLAS

4.1	Plantilla usada para los requisitos	16
4.2	Requisito funcional RF-001	17
4.3	Requisito funcional RF-002	17
4.4	Requisito funcional RF-003	17
4.5	Requisito funcional RF-004	17
4.6	Requisito funcional RF-005	18
4.7	Requisito funcional RF-006	18
4.8	Requisito funcional RF-007	18
4.9	Requisito funcional RF-008	18
4.10	Requisito funcional RF-009	19
4.11	Requisito funcional RF-010	19
4.12	Requisito funcional RF-011	19
4.13	Requisito funcional RF-012	19
4.14	Requisito funcional RF-013	20
4.15	Requisito funcional RF-014	20
4.16	Requisito funcional RF-015	20
4.17	Requisito funcional RF-016	20
4.18	Requisito funcional RF-017	21
4.19	Requisito funcional RF-018	21
4.20	Requisito funcional RF-019	21
4.21	Requisito funcional RF-020	21
4.22	Requisito funcional RF-021	22
4.23	Requisito funcional RF-022	22
4.24	Requisito no funcional NF-001	23
4.25	Requisito no funcional NF-002	23
4.26	Requisito no funcional NF-003	23
4.27	Requisito no funcional NF-004	24
4.28	Requisito no funcional NF-005	24

4.29	Requisito no funcional NF-006	24
4.30	Requisito no funcional NF-007	24
4.31	Requisito no funcional NF-008	25
4.32	Requisito no funcional NF-009	25
7.1	Estadísticas de la búsqueda	41
7.2	Resultados por tamaño de enjambre y espacio de búsqueda: porcentaje de aciertos, media y separación en cuartiles de pasos hasta el objetivo.	58

1. INTRODUCCIÓN

1.1. Motivación

La búsqueda de objetivos dentro de un cierto espacio en el menor tiempo posible, llamada Búsqueda en Tiempo Mínimo o MTS, por sus siglas en inglés, es un problema que se ha estudiado desde la Segunda Guerra Mundial por sus diversas aplicaciones tanto en operaciones militares como en misiones de búsqueda y rescate tras un desastre natural. A la hora de abordar este problema se han tomado diferentes enfoques como: modelar la búsqueda como un Proceso de Decisión de Markov Parcialmente Observable, donde la posición del objetivo es el estado desconocido y las observaciones son las mediciones realizadas por los sensores, lo que permite obtener una solución óptima al problema pero que resulta computacionalmente muy costosa; dentro de la teoría de control como un problema de control estocástico, donde se tiene en cuenta la información obtenida por los sensores para controlar el movimiento; o desde el punto de vista de la fusión sensorial, buscando un enfoque más realista.

Estas diferentes aproximaciones hacen necesaria una revisión de las distintas estrategias usadas desde estrategias tradicionales de fuerza bruta a otras más informadas.

1.2. Objetivos

Siguiendo lo expuesto en la sección anterior, este trabajo tiene como objetivo una comparación de varias estrategias de búsqueda. Para ello hemos establecido varias metas:

- **Desarrollar un framework** que permita simular el entorno de búsqueda y las acciones de los agentes. Que además incluirá una interfaz gráfica para mostrar el comportamiento de los agentes durante la búsqueda.
- **Comprobar la eficacia de varias estrategias** probando distintas configuraciones dentro del framework, realizando un análisis de las fortalezas de las distintas estrategias y comparando su rendimiento en diferentes situaciones.
- **Experimentación con la búsqueda multiagente** y, posiblemente, encontrar un límite a partir del cual aumentar el tamaño del enjambre no mejora los resultados de la búsqueda e incluso podría empeorarlos.

1.3. Entorno socio-económico

La búsqueda de objetivos ha sido siempre un campo de estudio de gran interés, creciendo mucho gracias a los avances en tecnologías de sensores y cámaras, drones, creando

cada vez agentes más capaces, además de avances en los algoritmos que controlan a estos agentes, el campo de investigación de operaciones (operations research) avanza con nuevos algoritmos basados en el estudio de hongos, colonias de hormigas o modelos matemáticos de optimización.

El ámbito socio-económico refleja una compleja intersección entre avances tecnológicos hacia aplicaciones civiles y las aplicaciones militares y en defensa de esta tecnología. En las aplicaciones civiles, el desarrollo de drones de búsqueda y rescate ha transformado las capacidades de respuesta a emergencias, reduciendo el coste y mejorando el tiempo de reacción en escenarios de rescates marítimos o búsqueda de personas desaparecidas. Esto se pudo ver en las operaciones de búsqueda realizadas por la UME tras el desastre de la DANA en Valencia el pasado de noviembre [1]. Los beneficios económicos del uso de estos drones son significativos: al realizar operaciones de búsqueda autónoma se puede reducir el coste de personal, minimizar la exposición al riesgo y permite operaciones de forma continuada 24/7 en condiciones en las que el uso de equipos humanos sería demasiado peligroso.

El impacto socio-económico fuera de lo militar no termina ahí, expandiéndose más allá de los servicios de emergencia civiles. En el sector agrónomo cada vez se incluye más el uso de sistemas de monitorización basados en drones junto con el uso de sistemas expertos para la vigilancia del estado de los cultivos o del ganado; o en la inspección de infraestructuras usando agentes autónomos para comprobar el estado de puentes, de líneas de suministro y de torres de alta tensión.

Las aplicaciones militares y en defensa de la tecnología de la búsqueda en tiempo mínimo constituyen el mayor motivador en la investigación y los desarrollos en este campo, aunque eso también hace que se encuentren clasificados la mayor parte de los trabajos. La recogida de inteligencia y las misiones de reconocimiento y de vigilancia se benefician mucho de sistemas autónomos capaces de buscar rápidamente en terrenos con incertidumbre y de localizar objetivos necesitando de forma mínima la supervisión de un humano. Los gobiernos han invertido mucho en el desarrollo de estas tecnologías debido a su valor estratégico. Esto crea una situación compleja en la que los desarrollos más punteros en optimización de algoritmos permanecen ocultos de los investigadores, y por lo que posibles avances son retrasados o completamente imposibilitados debido a las restricciones de acceso. Una de las aportaciones de este trabajo es el comienzo en el desarrollo de un framework para la simulación de estas búsquedas, disponible a la comunidad académica de forma libre y abierta.

Este trabajo está relacionado con el Objetivo de Desarrollo Sostenible número 9 relativo a *industria, innovación e infraestructura*[2]. En concreto, con la quinta de sus metas: “Aumentar la investigación científica y mejorar la capacidad tecnológica de los sectores industriales”, pudiendo ver el impacto a través del indicador **9.5.2**, ya que al hacer público un sistema de simulación para este tipo de operaciones, se permite la entrada a un gran número de nuevos investigadores en este campo.

1.4. Marco regulador

Recientemente en el sector de los drones y aeronaves no tripuladas está evolucionando rápidamente y, por tanto, surgen nuevas regulaciones. Concretamente podemos mirar al *Reglamento UE 2021/664*[3], publicado en 2021 en el que se establece un marco regulatorio a nivel europeo para ofrecer sistemas, servicios y procedimientos para un acceso seguro, eficiente y asequible al espacio aéreo, para operaciones que empleen UAVs, otros tipos de aeronaves operadas de forma autónoma o a distancia sin piloto a bordo y los sistemas que los controlan; a todos estos agentes que operan en estos espacios aéreos se les llama de forma general sistemas de aeronaves no tripuladas, *UAS*. Y el conjunto de sistemas y servicios ofrecidos a los sistemas se denomina *U-Space*. U-Space es una iniciativa en la que participan agencias tanto a nivel europeo como la Agencia Europea de Seguridad Aérea, EASA, como nivel nacional como el Ministerio de Transporte y Movilidad Sostenible o ENAIRE, el gestor de navegación aérea de España.

Un espacio aéreo U-Space es una zona en la que únicamente se permiten las operaciones con UASs que se deben realizar con el soporte de los servicios U-Space. Se contemplan 4 servicios obligatorios que deben prestarse dentro de un espacio aéreo U-Space:

- Servicio de Identificación de Red: que identifica a los operadores de los drones, aportando información sobre posición, trayectoria y rumbo de las aeronaves durante las operaciones.
- Servicio de Geoconsistencia: que ofrece información precisa y actualizada sobre las condiciones operacionales, limitaciones del espacio aéreo y restricciones temporales del espacio aéreo.
- Servicio de Autorización de Vuelo: que garantiza que las operaciones dentro del espacio aéreo U-Space se realizan sin entrar en conflicto con otros UASs y controlando que se cumplen las restricciones del espacio aéreo.
- Servicio de Información de Tráfico: que ofrece información a los operadores de tráfico, tripulado y no tripulado, que se encuentran en la proximidad.

Además de 2 servicios opcionales que también se pueden ofrecer:

- Servicio de Información Meteorológica: que ofrece datos meteorológicos relevantes para la ejecución de forma segura de las operaciones en la zona.
- Servicio de Supervisión de Conformidad: permite verificar que las operaciones de los UASs se realizan de acuerdo a los requisitos previamente establecidos.

Las capacidades de los sistemas descritos en este trabajo se podrían categorizar dentro de los Servicios de Identificación de Red y los Servicios de Supervisión de Conformidad.

Por lo que no se podría considerar un Proveedor de Servicios de U-Space, USSP, ya que para ello debería ofrecer los 4 servicios obligatorios.

1.5. Estructura

Este trabajo presenta la siguiente estructura, de forma que se puedan comprobar los objetivos expuestos:

2. **Estado del arte:** en este capítulo se realiza una revisión de otros trabajos relacionados con el problema de búsqueda en tiempo mínimo, mostrando algunos de los avances que se han realizado en este campo, además de introducir ciertos conceptos básicos de este campo.
3. **Modelado del problema:** donde se explica el problema definiendo sus características y la implementación de ciertos aspectos adicionales.
4. **Desarrollo del Framework:** capítulo en el que se exponen los requisitos definidos para el desarrollo del framework.
5. **Proceso de experimentación:** en este capítulo se detalla la metodología usada a la hora de realizar las pruebas y el propósito de las mismas.
6. **Análisis de los resultados:** capítulo en el que se explican y analizan, usando gráficas para visualizar los datos, los resultados obtenidos en las pruebas haciendo algunas comparaciones entre las estrategias de búsqueda.
7. **Conclusiones:** finalmente se desarrollarán las conclusiones, explicando qué objetivos se han completado, y se presentarán algunas posibles líneas de investigación para futuros trabajos en este campo.

2. ESTADO DEL ARTE

En la literatura existe gran variedad enfoques y soluciones para el problema de búsqueda de objetivos en espacios con incertidumbre. Este no es un análisis exhaustivo del estado del arte, sino una muestra de las características más relevantes para este trabajo. Para un análisis más profundo es recomendable el recorrido histórico del problema aportado en [4, Lanillos, 2013].

El problema de la búsqueda probabilística se inició tras la Segunda Guerra Mundial con la búsqueda del submarino estadounidense USS Scorpion en 1968, en la que se empleó un mapa de probabilidad para enfocar los esfuerzos en las zonas de mayor probabilidad, y la búsqueda de la bomba de hidrógeno de Palomares, en España, en 1966, donde se usaron las mismas técnicas. Desde entonces se han realizado numerosos avances en varios de los aspectos de este problema, entre ellos:

- **Planificadores.** Los avances en robótica y electrónica han permitido que los drones usados en las búsquedas tengan cada vez más potencia de cálculo, lo que les permite encontrar la ruta que minimice el tiempo esperado de búsqueda o maximice las posibilidades de localizar el objetivo. Dentro del campo de los planificadores se han desarrollado múltiples planificadores empleando distintas técnicas como: minimizar la entropía cruzada[5], el uso de algoritmos evolutivos[6]...
- **Controlador de Horizonte Recesivo.** Relacionado con los planificadores, uno de los primeros métodos para resolver el problema de búsqueda probabilística fue el uso de Procesos de Markov Parcialmente Observables[7]. El inconveniente era que esta solución no permitía escalar, a pesar de poder dar resultados óptimos. El Controlador de Horizonte Recesivo, RHC, permite establecer un horizonte temporal y calcular la secuencia óptima de movimientos dentro de este horizonte limitado. Así concatenando los horizontes podemos obtener una solución, que aunque subóptima, se acerca a la solución óptima del problema. Naturalmente cuanto más grande sea el horizonte mejor será la solución, pero esto conlleva que sea más costoso calcularla.
- **Búsqueda con múltiples UAVs.** El uso de múltiples agentes presenta ciertas ventajas, como posibles reducciones en el tiempo esperado para encontrar el objetivo. Pero esto depende de una buena coordinación entre los distintos agentes: o bien de forma descentralizada siendo los agentes los que se coordinan entre ellos, o bien mediante una Estación de Control en Tierra, GCS, coordinando de forma centralizada. Este enfoque centralizado presenta ciertas ventajas como un mayor control sobre el movimiento de los agentes; al ser un ente centralizado la información respecto a la búsqueda es más completa, frente a la creencia que pueda tener cada uno de los agentes; y se desacopla el desarrollo y mejora en las técnicas de planificación y control en tierra[8], del desarrollo de la tecnología de los UAVs.

3. MODELADO DEL PROBLEMA

El problema se ha modelado basándose en el modelo Global Deterministic Solution: Constraint Programming, descrito en [4], en el que se toma un enfoque de uso de restricciones, pudiendo formular el problema de la siguiente manera: partiendo de un espacio de búsqueda Ω , se discretiza sobre una malla G_Ω dividida en celdas rectangulares de tamaño $w_x \times w_y$. Sobre este espacio de búsqueda se tiene información a priori sobre la posición de uno o varios objetivos, asignando una distribución de probabilidad. Este área se sobrevuela con un conjunto de U UAVs para poder localizar los objetivos que se encuentran en esta. A lo largo de este capítulo se describen en mayor detalle los elementos que intervienen en este problema.

3.1. Generación del espacio de búsqueda

La búsqueda del objetivo no se hace completamente a ciegas, antes de empezar se parte de cierta información sobre la posible ubicación o ubicaciones donde es más probable que se encuentre el objetivo. Con esta información se crea un mapa de probabilidad, denotado como $b(v^t)$, que asigna a cada punto de Ω la probabilidad de que el objetivo se encuentre en esa posición en el instante de tiempo t . Se asume que el objetivo siempre está dentro de Ω y que se cumple que $\sum_{v^0 \in G_\Omega} P(v^0) = 1$.

Siguiendo ahora el modelo propuesto en [9], esta información se obtiene a partir de J fuentes de información, cada una aportando una capa de probabilidad $b_j(v^t)$, y combinándose todas las fuentes ponderando la creencia con un peso w_j .

A la hora de instanciar el problema y definir el mapa de probabilidad inicial $b(v^0)$, se discretiza Ω de forma uniforme en $w_x \cdot w_y$ celda formando una red G_Ω . Entonces para cada punto en la red, se suman las capas de probabilidad de las distintas fuentes, $b_j(v^0)$, ponderadas por su peso.

$$b(v^0) = \frac{1}{\sum_{j=1}^J w_j} \sum_{j=1}^J w_j \cdot b_j(v^0) \quad (3.1)$$

En nuestro caso cada fuente o indicio se modela como una distribución de probabilidad Gaussiana: generando una función de probabilidad alrededor de un punto τ y especificando una matriz de covarianza Σ . Entonces para cada punto $P = (x, y)$ de la capa de probabilidad inicial del indicio j , se calcula probabilidad de que el objetivo se encuentre en ese punto usando la siguiente formula:

$$b_{j,P}(v^0) = \frac{1}{2\pi \det(\Sigma)^{1/2}} \exp\left(-\frac{1}{2}(P - \tau)^T \cdot \Sigma^{-1} \cdot (P - \tau)\right) \quad (3.2)$$

Usando este modelo se pueden recrear escenarios muy variados como los escenarios de ejemplo en la figura 3.1: la primera imagen contiene 2 indicios de probabilidad, estableciendo que la probabilidad no se reparte igual entre ambos, el 70 % de la probabilidad se encuentra en uno de los indicios y el 30 % en el otro, esto se ha logrado asignando los pesos de 0.7 y 0.3 respectivamente a cada uno de los indicios; en la segunda se ha cambiado la matriz de covarianza para crear una distribución de probabilidad que no sea circular y dándole algo de distorsión.

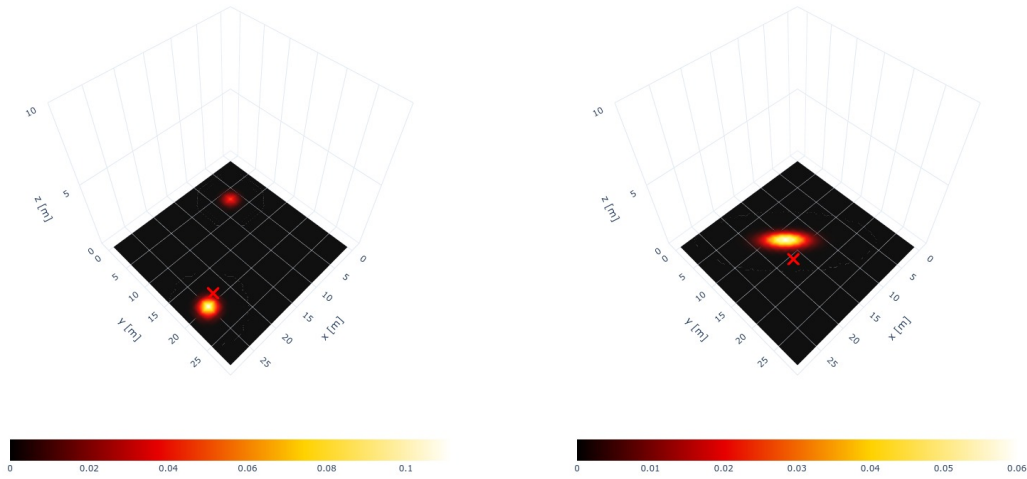


Fig. 3.1. Ejemplos de espacios de búsqueda posibles a generar

Una vez generado el espacio de probabilidad inicial, si no se especifica explícitamente una posición para el objetivo, esta se escoge de forma aleatoria alrededor de uno de los indicios, marcado con una cruz de color rojo en las ilustraciones. Usando este indicio se coloca el objetivo dentro de un intervalo de confianza de $n=3$, lo que equivale a un nivel de confianza del 99,74 %. Ya que asumimos que nuestras fuentes de información son fiables, el objetivo se encuentra no muy lejos de alguno de los indicios marcados por nuestras fuentes de información.

3.2. Modelado del objetivo

La posición del objetivo en el momento t se define como una variable aleatoria v^t . Esta variable indica que la posición del objetivo puede no ser fija ni es conocida con certeza, sino que la distribución de probabilidad de su posición está repartida por el espacio Ω .

Para modelar la información sobre la posición del objetivo se usa un modelo dinámico que describe la probabilidad de que el objetivo se desplace desde una posición en $t - 1$, v^{t-1} , a v^t dando un paso de tiempo de duración Δ , expresado como $P(v^{t-1}|v^t)$. Usando este modelo se puede actualizar la distribución de la creencia sobre la posición del objetivo a medida que se realiza la búsqueda. Este modelo se puede crear a partir de, por ejemplo,

las corrientes marinas y así poder tener en cuenta el movimiento que provocan sobre los objetivos cuando estos se encuentran flotando a la deriva.

Para describir este modelo dinámico del objetivo hemos usado un modelo basado en procesos de Markov. En un modelo de objetivo Markoviano, el objetivo no tiene memoria de sus posiciones anteriores y sus acciones solo están definidas por su estado actual. Usando la ecuación de Chapman-Kolmogorov [10], podemos calcular cuál será la distribución de probabilidad en el tiempo t , $\hat{b}(v^t)$ actualizando la creencia en $t - 1$, $b(v^{t-1})$, con el modelo dinámico del objetivo $P(v^t|v^{t-1})$:

$$\hat{b}(v^t) = \sum_{v^{t-1} \in G_\Omega} P(v^t|v^{t-1})b(v^{t-1}) \quad (3.3)$$

Este modelo de transición puede ser todo lo simple o complicado que queramos, incluyendo restricciones del entorno o del movimiento, como direcciones preferidas o límites de velocidad. En los espacios discretos el modelo de transición se puede implementar a través de una matriz de transición P_{kernel} que incluye la probabilidad de que el objetivo se mueva desde su posición actual a una de sus celdas vecinas. Para aplicar este modelo de objetivo basado en la matriz de transición, se usarán convoluciones, que permite actualizar y redistribuir las probabilidades hacia celdas vecinas según las probabilidades definidas en la matriz de transición P_{kernel} .

Un ejemplo de las matrices de transición que usamos sería la siguiente 3.4, una matriz de 3×3 , donde el valor central contiene la probabilidad de permanecer en la misma celda en que ya está (0.2), y el resto de valores (0.15 y 0.05) la probabilidad de moverse a las celdas vecinas en las direcciones cardinales o en diagonal.

$$P_{kernel} = \begin{bmatrix} 0,05 & 0,15 & 0,05 \\ 0,15 & 0,2 & 0,15 \\ 0,05 & 0,15 & 0,05 \end{bmatrix} \quad (3.4)$$

Como en este trabajo nos enfocamos en el caso de un objetivo estático, el modelo probabilístico de transición sería el siguiente:

$$P_{kernel} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \quad (3.5)$$

Aplicando este modelo estático para el objetivo, la ecuación 3.3 para actualizar la creencia sobre la posición del objetivo en el momento t a partir de la creencia en $t - 1$, quedaría así:

$$\hat{b}(v^t) = \cancel{P(v^t|v^{t-1})}b(v^{t-1}) = b(v^{t-1}) \quad (3.6)$$

3.3. Observaciones

Los UAVs tienen incorporado un sensor con el que son capaces de interactuar con el espacio de búsqueda. Utilizando este sensor realizan *observaciones*, denotando la observación realizada por el agente u en el momento t como z_u^t . Para simular el comportamiento de este sensor se usa la función de verosimilitud $P(z_u^t | v^t, s_u^t)$ que, teniendo en cuenta la posición del agente s_u^t y la posición del objetivo v^t en el momento t , devuelve la probabilidad que se dé una cierta observación. Usando este modelo la información de la creencia de la posición del objetivo se puede actualizar a medida que el espacio de búsqueda se explora.

En concreto, la función de verosimilitud que usaremos para actualizar la probabilidad es la 3.7, basada en un modelo muy usado en la literatura[11], y cuenta con 3 parámetros:

- P_{max} , la probabilidad máxima de detección del sensor cuando el objetivo está en el rango de detección.
- $dmax$, distancia máxima a la que el sensor puede detectar el objetivo, esta distancia marca el radio del área de detección del sensor (o el apotema en caso de que tenga forma poligonal).
- σ , sensibilidad a la distancia actúa como un factor de caída exponencial, haciendo decaer la probabilidad de detección más rápido cuanto mayor sea más.

Esta ecuación se aplica a todos los puntos del mapa, siendo d la distancia euclidiana entre la posición actual del agente s_u^t y el punto cuya creencia se está actualizando.

$$P(z_u^t | v^t, s_u^t) = P_{max} \cdot \exp\left(-\sigma \left(\frac{d}{dmax}\right)^2\right) \quad (3.7)$$

En el problema de búsqueda en espacios con incertidumbre se considera únicamente que estas observaciones serán: o bien de detección del objetivo, $z_u^t = D$, en cuyo caso la búsqueda terminará; o de no detección $z_u^t = \bar{D}$ y la búsqueda del objetivo continúa. Nosotros asumiremos un sensor ideal, que detectará siempre al objetivo si este está dentro del rango de detección. El rango de detección está marcado por la huella del sensor, la proyección sobre el suelo de la forma del sensor teniendo en cuenta la orientación de este. Nosotros modelaremos este área de detección como un cuadrado de tamaño constante posicionado siempre bajo el agente, inspirado en las cámaras que se usan en los drones reales.

Dado que en algunos casos vamos a trabajar con más de un agente para realizar la búsqueda, cada uno realizando sus propias observaciones, estas deben ponerse en común. Hemos optado por un enfoque centralizado, de forma que todos los agentes conocen la misma información en todo momento y nos es más fácil razonar sobre qué está ocurriendo durante la búsqueda.

3.4. Fusión de información: Filtro Recursivo Bayesiano

Si ahora ponemos en común este modelado del problema llegamos a un enfoque probabilístico que, tomando un modelo dinámico del objetivo y las mediciones de los sensores, devuelve la distribución de probabilidad con la posición del objetivo para el instante t . Este modelo es el *Filtro Bayesiano Recursivo* [12], (RBF por sus siglas en inglés) un algoritmo que consta de dos pasos que se repetirán hasta que el objetivo se detecte o cuando se alcance el límite temporal T .

Predicción: usando el modelo probabilístico de movimiento del objetivo se ajusta el mapa probabilidad. Usando la ecuación 3.3 podemos calcular la distribución de la posición del objetivo en el instante t , basándonos en el modelo del objetivo $P(v^t|v^{t-1})$ y la creencia $b(v^{t-1})$.

$$b(v^t) = \sum_{v^{t-1} \in G_\Omega} P(v^t|v^{t-1})b(v^{t-1})$$

Actualización: redistribuir la probabilidad a priori sobre el mapa a medida que el agente se mueve. Para calcular la creencia de la posición en el instante t se toma la creencia anterior y se actualiza usando la información obtenida de los sensores. En concreto, se usa la probabilidad de observación en el momento t , z_u^t , dado el estado del agente, s_u^t , y el del objetivo, v^t . Además se añade una constante de normalización, $\frac{1}{\xi}$, para que la suma de todas probabilidades sea 1, dando como resultado la siguiente expresión[4]:

$$\hat{b}(v^t) = \frac{1}{\xi} \prod_{u=1}^U P(z_u^t|v^t, s_u^t) b(v^t) \quad (3.8)$$

El pseudocódigo del algoritmo se muestra en 1, el cual se inspira en el trabajo de [13]:

Algorithm 1 Pseudocódigo del RBF

Require: $P(v^0), P(v^t|v^{t-1}), P(z_u^t|v^t, s_u^t)$ ▶ Initial belief, target dynamic and sensor models

Require: $s_{1:U}^{0:T}, z_{1:U}^{0:T}, T, \Delta T$ ▶ Sensor measurements, UAVs states, search time, time interval

- 1: $b(v^0) = \frac{1}{\xi} \prod_{u=1}^U P(z_u^0|v^0, s_u^0)P(v^0)$ ▶ Initialize target belief
 - 2: $N = T/\Delta T$ ▶ Number of time steps
 - 3: **for** $t = 1 : N$ **do**
 - 4: $\hat{b}(v^t) = \sum_{v^{t-1} \in G_\Omega} P(v^t|v^{t-1})b(v^{t-1})$ ▶ Prediction Step
 - 5: $b(v^t) = \frac{1}{\xi} \prod_{u=1}^U P(z_u^t|v^t, s_u^t)\hat{b}(v^t)$ ▶ Update Step
 - 6: **end for**
 - 7: **return** $b(v^N)$ ▶ Updated target belief up to $t = N$
-

3.5. Ejemplo ilustrativo del RBF

Veamos este algoritmo en acción 3.2, usaremos una cuadrícula de 3×3 con una distribución de probabilidad inicial arbitraria. Un agente se moverá a través de esta recreando el siguiente camino $s^{0:2} = \{0, 3, 4\}$, empezando en la esquina superior izquierda y yendo al centro. En la figura se marca con un borde azul la celda en la que se encuentra el agente y, además de numerar las celdas, se muestra cual es el valor de $b(v^t)$ en cada una de ellas.

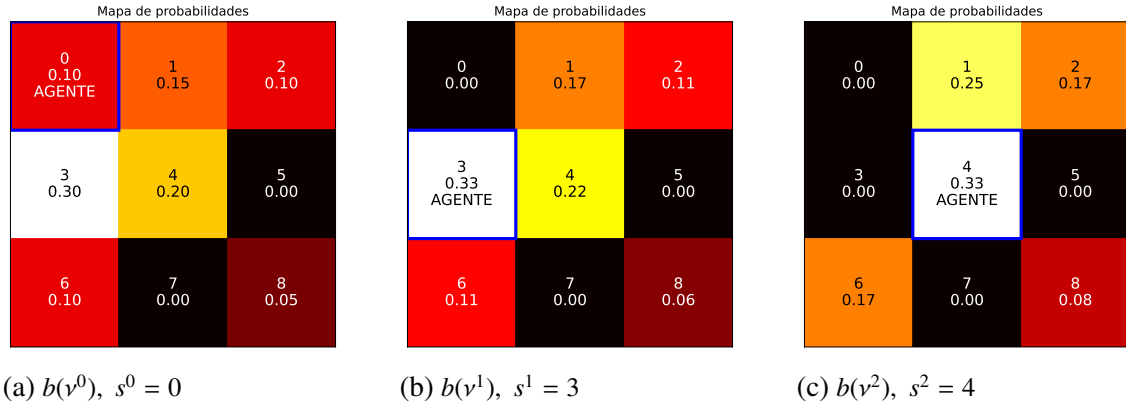


Fig. 3.2. Ejemplo ilustrativo del Filtro Recursivo Bayesiano

En la figura 3.2 se muestra cómo a medida que el agente se mueve y realiza las observaciones, la distribución de probabilidad se va actualizando para que se cumpla que la suma de la probabilidad de todas las celdas de la malla sea 1, cumpliendo con la hipótesis de partida en la que el objetivo se encuentra dentro de la región de búsqueda.

3.6. Notación

A continuación, se presenta un resumen de la notación utilizada en este capítulo y se continuará usando a lo largo del trabajo, ordenada por orden de aparición:

- Ω : Área de búsqueda.
- G_Ω : Conjunto de celdas que representan la cuadrícula (malla) del área de búsqueda Ω discretizada.
- w_x, w_y : Dimensiones de la malla G_Ω .
- U : Número de UAVs que participan en la búsqueda.
- v^t : Posición del objetivo en el instante t , modelada como una variable aleatoria.
- $b(v^t) = P(v^t)$: Mapa de probabilidad que describe la creencia sobre la ubicación del objetivo en el tiempo t .

- $b(v^0)$: Probabilidad inicial del objetivo en cada celda, combinando diferentes fuentes de información.
- $b_j(v^0)$: Capa de probabilidad de la fuente de información j .
- w_j : Peso asignado a la fuente de información j .
- $P(v^t|v^{t-1})$: Modelo dinámico del objetivo, que describe la probabilidad de transición del objetivo de v^{t-1} a v^t en un intervalo de tiempo Δ .
- $P(z_u^t|v^t, s_u^t)$: Modelo del sensor, que describe la probabilidad de obtener una medición z_u^t dada la posición del objetivo v^t y la posición del agente s_u^t .
- z_u^t : Medición del sensor a bordo del UAV u en el instante t , con valores $z_u^t = D$ (detección) o $z_u^t = \bar{D}$ (no detección).
- s_u^t : Posición del UAV u en el tiempo t .
- c_u^t : Acción de control aplicada por el UAV u en el tiempo t .
- $c_u^t \in \{N, NE, E, SE, S, SW, W, NW\}$: Conjunto de direcciones cardinales y diagonales que el UAV puede tomar como acciones de control.
- Δ : Intervalo de tiempo entre dos mediciones consecutivas.
- $\hat{b}(v^t)$: Creencia predicha sobre la ubicación del objetivo en el tiempo t .
- ξ : Factor de normalización que asegura que $\sum_{v^t \in G_\Omega} b(v^t) = 1$.
- P_{kernel} : Kernel bidimensional que representa las probabilidades de transición hacia celdas vecinas.
- $\tilde{b}(v^t)$: Mapa de probabilidad actualizado de la ubicación del objetivo v^t utilizando un enfoque bayesiano recursivo sin normalización.
- $P_d(s_{1:U}^{0:N})$: Probabilidad de detección, definida como la probabilidad de detectar al menos una vez el objetivo durante el horizonte de tiempo N .
- h_u^t : Altitud del UAV u en el instante t .
- x_i^t : Posible localización del objetivo en el área de búsqueda en el instante de tiempo t , representada por la partícula i .
- w_i^t : Peso asociado a la partícula i en el instante de tiempo t .

4. PROPUESTA DE TRABAJO

La simulación del comportamiento de los agentes durante la búsqueda es fundamental para el tipo de experimentación que queremos realizar. Esta simulación se hará a través de un framework que desarrollaremos explícitamente para este propósito. Concretamente, queremos crear una forma de simular el comportamiento de agentes de acuerdo al Filtro Bayesiano Recursivo, explicado en 3. Además, ya que buscamos comparar varias estrategias, debemos implementar varias dinámicas de movimiento para los agentes: búsqueda por fuerza bruta barriendo el espacio de acuerdo a algún patrón fijo y planificadores voraces guiados por heurísticas.

Antes de exponer todos los requisitos que debe cumplir este simulador, exploremos algunos de los principales problemas con los que este framework debe lidiar.

4.1. Problema de la miopía

Un planificador voraz, es un modelo de dinámica que guiado por una heurística es capaz de decidir cuál de sus acciones **inmediatas**, le lleva a un mejor estado. En los planificadores voraces se puede producir un fenómeno denominado miopía, en el que, por falta de información o porque el horizonte de decisión es muy pequeño y no son capaces de planificar a largo plazo, el agente toma decisiones que, aunque óptimas en el corto plazo, obtienen resultados subóptimos a la larga. Este problema se ilustra excelentemente en el trabajo de P. Lanillos et al. *Minimum Time Search of Moving Targets in Uncertain Environments*[14]:

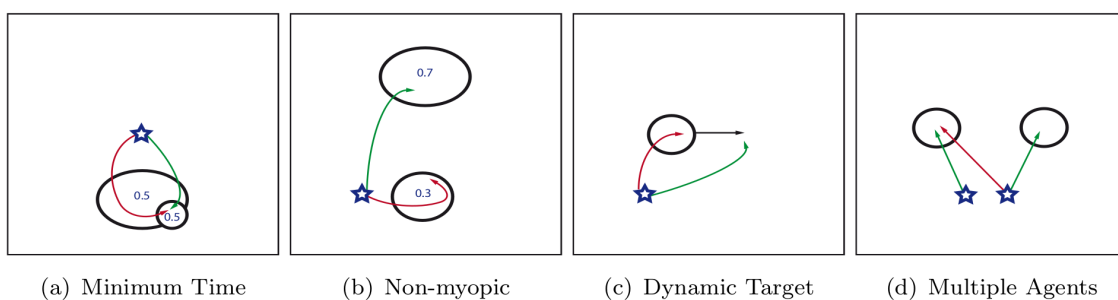


Fig. 4.1. Ejemplos ilustrativos de la miopía, a partir de la tesis de P. Lanillos et al. [14]

En estas imágenes podemos ver representados varios escenarios ilustrativos de un comportamiento miope. Las flechas de color representan las trayectorias realizadas por los agentes, dibujados como estrellas. La trayectoria verde representa una solución óptima que maximiza la probabilidad de encontrar el objetivo en el menor tiempo posible, y la trayectoria roja es una trayectoria miope y por tanto subóptima. En los casos (b) y (d) se puede ver especialmente bien cómo los agentes, debido a decisiones óptimas en el corto

plazo, no encuentran la solución que maximizaría la probabilidad de detección a la larga. En el caso (b) el agente miope decide buscar en una zona cercana pero con una menor probabilidad de encontrar el objetivo, cuando la estrategia optima sería ir a la zona de mayor probabilidad.

Basados en un criterio de corrección de miopía[9], implementamos una heurística que guía a los agentes hacia las zonas de mayor probabilidad, aquellas donde es más probable que se encuentre el objetivo. La heurística toma la distancia entre la posición del agente s_u^t y el punto de mayor probabilidad en el momento t , τ^t , y la ajusta dependiendo del peso que queramos darle

$$MYOP = \sum_{v^t \in G_\Omega} \prod_{u=1}^U H(\tau^t, s_u^t) b(v^t) \quad (4.1)$$

$$H(\tau^t, s_u^t) = 1 - \lambda^{dist(\tau^t, s_u^t)} \quad \text{con} \quad 0 < \lambda < 1 \quad (4.2)$$

Podemos ver este criterio de reducción de la miopía en acción en una situación reconstruida parecida a la que vimos en el caso (b). Las zonas de probabilidad tienen un 30 % y un 70 % respectivamente. Podemos ver que el agente miope (en la izquierda) busca en la zona de menor probabilidad, mientras que el agente usando el criterio de corrección de la miopía, busca allá donde la probabilidad sea mayor.

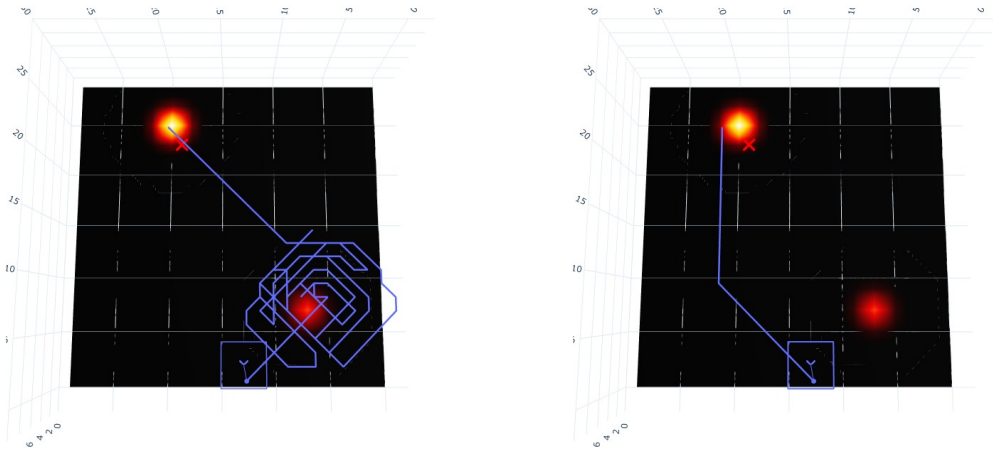


Fig. 4.2. Comparación del funcionamiento entre agente miope y un agente con la miopía reducida

4.2. Coordinación de varios agentes

Una de las principales ventajas que ofrecen los UAVs es que se pueden controlar de forma remota y formar grandes grupos que se coordinan para buscar de forma organizada. Uno de los objetivos de este trabajo requiere la coordinación de varios agentes en la

realización de las búsquedas, por lo que deben poder interactuar entre ellos y compartir información. En nuestro caso hemos usado un modelo idealizado de GCS donde los agentes pueden recibir y enviar información de forma instantánea con el GCS, de forma que existe un único mapa de creencia sobre la posición del objetivo que todos comparten. Durante la búsqueda, al usar el algoritmo del RBF, los agentes se pueden apoyar en las observaciones realizadas por los demás y se consigue una coordinación más eficiente. Gracias a esta coordinación global podrían emerger de forma espontánea comportamientos como la separación del área de búsqueda en *zonas*, cada una investigada por distintos agentes. En un escenario real esto no podría funcionar así, pero de la misma forma que un RHC periódicamente se actualiza para crear un nuevo horizonte de búsqueda, los UAVs se podrían comunicar con la estación de control de forma periódica para actualizar su creencia con los datos aportados por el resto de agentes.

4.2.1. Evitar colisiones

Para añadir mayor grado de realismo y hacer que estos resultados sean comparables con otros trabajos en la búsqueda en los que se emplean UAVs, se introdujo una restricción de penalización de colisiones entre UAVs[9]. Para evitar la colisión de los agentes se definió una distancia mínima de separación que debían compartir todos los agentes. Si cualquiera de las posibles posiciones disponibles desde la actual violaba esta distancia, esa posición se penalizaba infinitamente.

4.3. Requisitos

Como en todo proyecto de software, antes de empezar a desarrollarlo, debemos definir las capacidades que el sistema y los distintos componentes que lo constituyen deben cumplir para poder alcanzar los objetivos propuestos. Estos requisitos se pueden dividir en funcionales y no funcionales.

Los requisitos funcionales son aquellos que especifican funciones que el sistema debe poder realizar. Un ejemplo de estas acciones sería simular la búsqueda de un objetivo usando un enjambre con una cantidad arbitraria de drones.

Los requisitos no funcionales, en cambio, describen cómo el sistema debe comportarse y las cualidades que debe poseer como el rendimiento, la escalabilidad o la seguridad del código. Este tipo de requisitos se enfocan en cómo debe funcionar el software y no en qué debe poder hacer.

4.3.1. Plantilla de requisitos

A la hora de especificar los requisitos utilizaremos la tabla 4.1 a modo de plantilla rellenando los distintos campos con la información relevante. Estos campos son:

- **Identificador:** Para referenciar de forma única cada requisito. Siguiendo el formato RF-XXX para los requisitos funcionales y NF-XXX para los requisitos no funcionales.
- **Nombre:** Título que describe de forma general el requisito.
- **Fuente:** Indica cual es el origen del requisito. Puede ser **sistema** o **analista**. Los requisitos elicitados por el sistema
- **Categorías:** Indica el grupo o grupos a los que pertenece el requisito. Estas categorías son distintas entre los requisitos funcionales y no funcionales, y se les aplicara un identificador numérico que se especificará al inicio de cada una de las secciones dedicadas a los requisitos.
- **Necesidad:** Indica el grado de importancia que supone la implementación del requisito. Pudiendo ser el requisito **Esencial**, **opcional** o **deseable**
- **Prioridad:** Indica el nivel de urgencia con el que se debe completar el requisito. Puede ser **alta**, **media** o **baja**.
- **Verificabilidad:** Muestra el grado de facilidad con el que se puede comprobar que el sistema cumple con el requisito. Pudiendo ser **alta**, **media** y **baja**.
- **Estabilidad:** Indica cómo de probable es que el requisito cambie en el futuro. Puede ser **estable** o **inestable**.
- **Dependencias:** Indica qué requisitos es necesario completar antes de poder implementar el requisito actual.
- **Descripción:** Donde se describe de forma detallada el requisito.

Identificador		Nombre			
Fuente		Categorías		Necesidad	
Prioridad		Verificabilidad		Estabilidad	
Dependencias					
Descripción					

TABLA 4.1. PLANTILLA USADA PARA LOS REQUISITOS

4.3.2. Requisitos funcionales

En esta sección se exponen las tablas con los requisitos funcionales identificados para el framework. Las categorías usadas para clasificar los requisitos son: Algoritmos de búsqueda, 001; Proceso de búsqueda, 002; Interfaz Gráfica, 003; Configuración del problema, 004; y Análisis de resultados, 005.

Identificador	RF-001	Nombre	Búsqueda de Cortacesped		
Fuente	Sistema	Categorías	001	Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias					
Descripción	El sistema debe permitir simular la búsqueda de un objetivo haciendo que el agente siga un patrón de cortacesped				

TABLA 4.2. REQUISITO FUNCIONAL RF-001

Identificador	RF-002	Nombre	Búsqueda de Cuadrado creciente		
Fuente	Sistema	Categorías	001	Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias					
Descripción	El sistema debe permitir simular la búsqueda de un objetivo haciendo que el agente siga un patrón de cuadrado creciente				

TABLA 4.3. REQUISITO FUNCIONAL RF-002

Identificador	RF-003	Nombre	Búsqueda con heurísticas		
Fuente	Sistema	Categorías	001	Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias					
Descripción	El sistema debe permitir simular la búsqueda de un objetivo estando el agente guiado por una heurística				

TABLA 4.4. REQUISITO FUNCIONAL RF-003

Identificador	RF-004	Nombre	Mostrar comportamiento de los agentes		
Fuente	Sistema	Categorías	002	Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias	RF-021,				
Descripción	El sistema debe poseer una forma de mostrar el comportamiento de los agentes durante la búsqueda				

TABLA 4.5. REQUISITO FUNCIONAL RF-004

Identificador	RF-005	Nombre	Exportar imágenes		
Fuente	Sistema	Categorías	002	Necesidad	Opcional
Prioridad	media	Verificabilidad	Alta	Estabilidad	Estable
Dependencias					
Descripción	El sistema debe permitir guardar las imágenes generadas por la interfaz gráfica				

TABLA 4.6. REQUISITO FUNCIONAL RF-005

Identificador	RF-006	Nombre	Búsqueda multi-agente		
Fuente	Sistema	Categorías	001	Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias					
Descripción	El sistema debe poder simular la búsqueda de un objetivo permitiendo el uso de una cantidad arbitraria de agentes				

TABLA 4.7. REQUISITO FUNCIONAL RF-006

Identificador	RF-007	Nombre	Posiciones de inicio aleatorias para los agentes		
Fuente	Sistema	Categorías	003	Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias	RF-011,				
Descripción	El sistema debe permitir crear escenarios de búsqueda de forma procedural especificando ciertos parámetros				

TABLA 4.8. REQUISITO FUNCIONAL RF-007

Identificador	RF-008	Nombre	Tamaño variable del espacio de búsqueda		
Fuente	Sistema	Categorías	003	Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias	RF-011,				
Descripción	El sistema debe permitir crear escenarios de búsqueda de tamaño arbitrario				

TABLA 4.9. REQUISITO FUNCIONAL RF-008

Identificador	RF-009	Nombre	Indicios en el espacio de búsqueda		
Fuente	Sistema	Categorías	003,004	Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias					
Descripción	El sistema debe permitir crear escenarios de búsqueda a partir de unos indicios sobre la posición del objetivo				

TABLA 4.10. REQUISITO FUNCIONAL RF-009

Identificador	RF-010	Nombre	Resultados de la búsqueda		
Fuente	Sistema	Categorías	005	Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias					
Descripción	El sistema debe condensar la información de la búsqueda para poder realizar un análisis de los datos, en una tabla o un formato similar				

TABLA 4.11. REQUISITO FUNCIONAL RF-010

Identificador	RF-011	Nombre	Archivos de configuración		
Fuente	Sistema	Categorías	003	Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias					
Descripción	El sistema debe poder recrear los escenarios de búsqueda que se le den a través de un archivo de configuración				

TABLA 4.12. REQUISITO FUNCIONAL RF-011

Identificador	RF-012	Nombre	Validación de la entrada		
Fuente	Sistema	Categorías	004	Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias					
Descripción	El sistema debe comprobar que los parámetros con los que va a trabajar son válidos. Se usarán asserts para asegurar que estos parámetros son válidos.				

TABLA 4.13. REQUISITO FUNCIONAL RF-012

Identificador	RF-013	Nombre	Selección de la posición del objetivo		
Fuente	Sistema	Categorías	004	Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias	RF-011, RF-019				
Descripción	El sistema debe colocar el objetivo en el espacio de búsqueda cerca de alguno de los indicios especificados o en la posición especificada en la configuración				

TABLA 4.14. REQUISITO FUNCIONAL RF-013

Identificador	RF-014	Nombre	Mostrar estado del espacio de búsqueda		
Fuente	Sistema	Categorías	002, 003	Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias					
Descripción	El sistema debe mostrar cómo se actualiza el estado del mapa de probabilidad de creencia a medida que se realiza el proceso de búsqueda				

TABLA 4.15. REQUISITO FUNCIONAL RF-014

Identificador	RF-015	Nombre	Detección del objetivo		
Fuente	Sistema	Categorías	002	Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias	RF-020				
Descripción	El sistema debe parar la simulación cuando se encuentre el objetivo				

TABLA 4.16. REQUISITO FUNCIONAL RF-015

Identificador	RF-016	Nombre	Compartir información entre agentes		
Fuente	Sistema	Categorías	002	Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias					
Descripción	Durante el proceso de búsqueda los agentes deben poder comunicar información entre ellos, como el estado del mapa de probabilidad o su posición				

TABLA 4.17. REQUISITO FUNCIONAL RF-016

Identificador	RF-017	Nombre	Distancia mínima entre agentes		
Fuente	Sistema	Categorías	002	Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias	RF-006, RF-011				
Descripción	El sistema debe asegurar que los agentes nunca se acercan más de una distancia mínima, por el bien de los equipos en una situación real				

TABLA 4.18. REQUISITO FUNCIONAL RF-017

Identificador	RF-018	Nombre	Mostrar el estado del objetivo		
Fuente	Sistema	Categorías		Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias	RF-013				
Descripción	El sistema debe mostrar en la interfaz gráfica la posición del objetivo durante la búsqueda				

TABLA 4.19. REQUISITO FUNCIONAL RF-018

Identificador	RF-019	Nombre	Distribución probabilística para los indicios		
Fuente	Sistema	Categorías	004	Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias	RF-011				
Descripción	El sistema debe generar el espacio de búsqueda inicial a partir de la distribución probabilística especificada para cada indicio				

TABLA 4.20. REQUISITO FUNCIONAL RF-019

Identificador	RF-020	Nombre	Simulación del funcionamiento del sensor		
Fuente	Sistema	Categorías	002	Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias	RF-011				
Descripción	El sistema debe simular el comportamiento del sensor a bordo de los agentes, usando los parámetros especificados				

TABLA 4.21. REQUISITO FUNCIONAL RF-020

Identificador	RF-021	Nombre	Mostrar funcionamiento del sensor		
Fuente	Sistema	Categorías	003	Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias					
Descripción	El sistema debe mostrar en la interfaz gráfica el comportamiento del sensor de cada uno de los agentes				

TABLA 4.22. REQUISITO FUNCIONAL RF-021

Identificador	RF-022	Nombre	Proceso determinista		
Fuente	Sistema	Categorías	003, 005	Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias					
Descripción	El sistema debe tener un comportamiento determinista para la reproducibilidad de los resultados a través de una semilla para los procesos aleatorios				

TABLA 4.23. REQUISITO FUNCIONAL RF-022

4.3.3. Requisitos no funcionales

En esta sección exponemos los requisitos no funcionales que el sistema debe cumplir. Las categorías usadas para clasificar estos requisitos no funcionales son: Rendimiento, 001; Mantenibilidad, 002; Usabilidad, 003; y Eficiencia, 004.

Identificador	NF-001	Nombre	Velocidad a la hora de calcular		
Fuente	Sistema	Categorías	001	Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias					
Descripción	El sistema debe estar optimizado para ejecutar las búsquedas en el menor tiempo posible. Para un único agente debería tardar menos de 1 minuto en finalizar la simulación y el tiempo debería aumentar acorde a la escala con la que se modifica la situación: si el espacio de búsqueda se duplica, ya que el área de búsqueda se cuadriplica, el tiempo máximo también aumentaría como mucho por un facto de 4; y al introducir 99 agentes más, el tiempo de calculo máximo aceptable 100 veces el original.				

TABLA 4.24. REQUISITO NO FUNCIONAL NF-001

Identificador	NF-002	Nombre	API clara		
Fuente	Sistema	Categorías	002	Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias					
Descripción	Los distintos módulos deben de poseer una API clara y estable a lo largo del tiempo				

TABLA 4.25. REQUISITO NO FUNCIONAL NF-002

Identificador	NF-003	Nombre	Interfaz gráfica intuitiva		
Fuente	Sistema	Categorías		Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias					
Descripción	El sistema debe poseer una interfaz gráfica intuitiva donde se pueda observar el proceso de búsqueda				

TABLA 4.26. REQUISITO NO FUNCIONAL NF-003

Identificador	NF-004	Nombre	Eficiencia en la búsqueda		
Fuente	Sistema	Categorías	004	Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias					
Descripción	El sistema debe realizar un uso eficiente de los recursos hardware y software para mejorar el rendimiento durante la búsqueda				

TABLA 4.27. REQUISITO NO FUNCIONAL NF-004

Identificador	NF-005	Nombre	Generar casos de prueba		
Fuente	Sistema	Categorías		Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias	RF-011				
Descripción	Diseñar un sistema de generación de casos de prueba para poder realizar muchas pruebas en distintas configuraciones				

TABLA 4.28. REQUISITO NO FUNCIONAL NF-005

Identificador	NF-006	Nombre	Documentación		
Fuente	Sistema	Categorías	003	Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias					
Descripción	Proveer documentación detallada sobre todas las funcionalidades del software.				

TABLA 4.29. REQUISITO NO FUNCIONAL NF-006

Identificador	NF-007	Nombre	Código modular		
Fuente	Sistema	Categorías	002	Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias					
Descripción	Diseñar el código de forma modular y siguiendo buenas prácticas de programación para facilitar el mantenimiento y la extensión de las funcionalidades del sistema				

TABLA 4.30. REQUISITO NO FUNCIONAL NF-007

Identificador	NF-008	Nombre	Sensible defaults		
Fuente	Sistema	Categorías	003	Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias					
Descripción	El sistema debe proveer valores por defecto que permitan un funcionamiento aceptable en la mayor parte de los casos				

TABLA 4.31. REQUISITO NO FUNCIONAL NF-008

Identificador	NF-009	Nombre	Interfaces comunes		
Fuente	Sistema	Categorías	003	Necesidad	Esencial
Prioridad	Alta	Verificabilidad	Alta	Estabilidad	Estable
Dependencias					
Descripción	Las funciones deben poseer interfaces comunes para poder intercambiar los métodos que conforman un caso de prueba por otros de la misma categoría				

TABLA 4.32. REQUISITO NO FUNCIONAL NF-009

5. DESARROLLO DEL FRAMEWORK

Con el objetivo de poder simular las acciones de los agentes en situaciones diversas hemos desarrollado una librería de Python que nos permite realizar las simulaciones especificando las características que nos interesaban para describir el problema. En esta sección explicaremos las funcionalidades desarrolladas de este framework, que ha sido fundamental para el desarrollo del trabajo, y una parte importante del esfuerzo dedicado.

5.1. Tecnologías y Frameworks

Para la realización del proyecto se ha usado el lenguaje Python por ser popular en el desarrollo de software y con una amplia selección de librerías para realizar estudios y análisis de datos, además de la flexibilidad en paradigmas que acepta, pudiendo optar por un diseño más parecido al lenguaje C, una aproximación orientada a objetos (OOP) o una mezcla de estas. Para garantizar buenas prácticas de programación, se han seguido las guías de estilo para Python, PEP 8, usando herramientas que lo hacen cumplir como el formateador *Black*.

La carga computacional que conlleva realizar las pruebas es elevada, aunque cada prueba no sea demasiado costosa, pudiendo ejecutarse al completo y generar la visualización en menos de 1 minuto; y realizándose el desarrollo del código del proyecto en un Lenovo Thinkpad T460s con 8 GB de memoria RAM, 4 núcleos y 256 GB de capacidad de almacenamiento en el disco duro. Es la cantidad de estas la que lo hace tan costoso, para cada caso de prueba se realizaron unas 4000 ejecuciones. Debido al gran coste computacional, la mayor parte de las ejecuciones se han realizado en el entorno de Python de Google Colaboratory. Permitiendo ejecutar cuadernos de Jupyter Notebook, con la ventaja de unir la ejecución de los programas con la documentación de estos y la inclusión de imágenes como parte del output, que hacen que sean herramientas muy útiles para trabajos como este de análisis de datos. Aunque con la desventaja de tener un tiempo de ejecución máximo de 12 horas por sesión y las posibles desconexiones por inactividad de los navegadores web.

Se ha usado también Google Drive para almacenar los archivos de configuración y los resultados de cada prueba. Gracias a la universidad, las cuentas educativas cuentan con 50 GB de almacenamiento aunque a penas se ha usado 1GB de almacenamiento.

Además del almacenamiento en Google Drive, se ha usado el sistema de control de versiones Git en la plataforma GitHub, creando un repositorio donde se encuentra todo el código [15].

5.1.1. Librerías

En cuanto a las librerías de Python usadas para el desarrollo del proyecto, estas son:

- **numpy**: Librería diseñada para realizar cálculos numéricos rápida y eficientemente, permitiendo usar vectores y matrices[16].
- **pandas**: Librería para el análisis de datos y la manipulación de estos usando tablas, llamadas *dataframes*[17].
- **matplotlib**: Módulo para crear gráficas con Python, se integra muy bien con las dos anteriores, además de ofrecer una gran variedad en las gráficas que se pueden crear[18].
- **plotly (y sus dependencias)**: biblioteca para crear gráficas más complejas que matplotlib, ofreciendo la posibilidad de crear visualizaciones de datos 3D y animaciones complejas[18].
- **scipy**: Librería para el análisis de datos que implementa gran cantidad de algoritmos matemáticos para realizar estadística, optimización interpolación, aprendizaje automático entre otros. Se ha usado principalmente a la hora de aproximar las funciones de densidad de probabilidad en el análisis de los resultados [18].
- **csv**: Librería para la lectura y escritura de archivos en formato CSV, que se han usado para guardar los resultados de las pruebas [19].
- **json**: Librería de Python para usar archivos en formato JSON [20].
- **os**: El módulo nativo de Python para interactuar con el sistema operativo, con la que podemos crear archivos o directorios o ejecutar comandos.
- **subprocess**: Módulo de Python para interactuar con la línea de comandos permitiendo más control que el que ofrece os.
- **drive**: Librería específica de Google Colab que de forma similar a os, nos ha permitido interactuar con el sistema de archivos de Google Drive sin depender de la interfaz web [21].

El framework desarrollado se puede dividir en 4 módulos principales como podemos observar en la Figura 5.1, dedicados cada uno a una tarea en específico: generación de los casos de prueba, implementación de los algoritmos de búsqueda, configuración inicial del problema de búsqueda y visualización de los resultados. A lo largo de este capítulo describiremos estos módulos, hablando de sus métodos y de las decisiones de diseño realizadas.

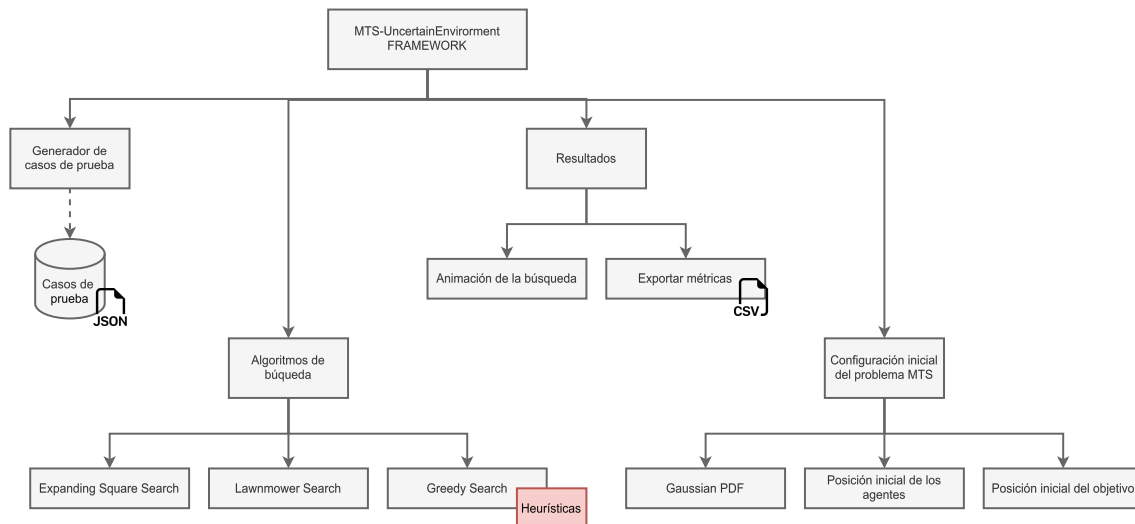


Fig. 5.1. Módulos principales del framework

5.2. Generación de los casos de pruebas

A la hora de crear escenarios de búsqueda para la simulación se usan unos archivos de configuración en los que se definen las características del problema, entre ellas: la posición inicial de los agentes, la disposición de los indicios en el espacio de búsqueda, una semilla para permitir que las pruebas sean reproducibles o los parámetros usados para recrear el comportamiento de los sensores durante la simulación.

Estos archivos de configuración se codifican en formato JSON, ya que es un formato fácil de utilizar, estandarizado y muy conocido por lo que cuenta con una gran integración con lenguajes de programación y librerías, más legible que otros formatos como CSV o XML, y al ser ficheros de texto simple se pueden leer y modificar usando cualquier editor de texto o IDE sin necesidad de herramientas externas.

Gracias a estos archivos de configuración podíamos crear escenarios específicos para realizar comprobaciones especiales o generarlos en grandes cantidades de forma rápida y automatizada para comparar los distintos algoritmos.

5.2.1. Parámetros de los casos de prueba

Los parámetros usados en estos archivos de configuración son los siguientes:

- **version:** Un número de versión que indica la versión actual de archivo de configuración que se usa, actualmente la versión es 1.1. Generalmente se ha intentado que el framework sea retrocompatible con versiones anteriores del mismo, y para ello se han introducido en muchas de las funciones parámetros por defecto y comprobación de errores.
- **size:** Una tupla con el tamaño del espacio de búsqueda.

- **cov**: La matriz o lista de matrices de covarianza usadas para calcular la probabilidad inicial del espacio de búsqueda. Representadas las matrices en el código como una lista de listas.
- **indicios**: La posición de cada indicio dentro del espacio de búsqueda.
- **pesos**: Lista de pesos asociados a cada indicio.
- **obj_pos**: Posición donde se encuentra inicialmente el objetivo.
- **p_transicion**: Kernel con las probabilidades de movimiento del objetivo en cada dirección.
- **indicio**: Indicio escogido para posicionar el objetivo, en caso de que no se haya especificado una posición directamente.
- **semilla**: Semilla para inicializar el generador de números aleatorios y que así las pruebas puedan replicarse más adelante si se quiere ver qué ha ocurrido para alguna en particular.
- **num_agents**: Número de agentes que se simularán y participarán en la búsqueda.
- **height**: Altura a la que vuelan los agentes. En este trabajo no se ha tenido en cuenta a la hora de encontrar el objetivo, usándose únicamente en la interfaz gráfica pero se ha incluido para poder usar en otros trabajos.
- **init_pos**: Lista con las posiciones iniciales de los agentes.
- **mov_delta**: Distancia que los agentes recorren al dar un *paso*.
- **posible_moves**: Posibles movimientos que pueden realizar los agentes en cada paso. Acepta las opciones: *cruz*, que permite al agente moverse en las 4 direcciones cardinales traducido a $c_u \in \{N, E, S, W\}$; y *8 dir*, añadiendo la posibilidad de moverse en diagonales, $c_u \in \{N, NE, E, SE, S, SW, W, NW\}$. Estas opciones de movimiento se pueden ampliar fácilmente para permitir otros tipos de movimiento.
- **min_dist**: Distancia mínima que deben mantener los agentes entre sí.
- **lambda**: Parámetro usado a la hora de definir el criterio de corrección de la miopía, de acuerdo a 4.2.
- **pdmax, dmax, sigma**: Parámetros usados para simular el sensor de los agentes, tal y como se explica en la sección 3.3.
- **num_steps**: Cantidad de pasos que pueden dar los agentes. Marca la duración total de la búsqueda.
- **itersteps**: Tamaño del horizonte temporal. Debido a que hemos usado algoritmos voraces, esta funcionalidad no se ha usado.

- **separation:** Separación entre los tramos en la trayectoria realizada por los algoritmos de fuerza bruta.
- **plan:** Ignorar la condición de parada al encontrar el objetivo, haciendo unicamente la planificación de la trayectoria.

5.3. Algoritmos de búsqueda

A pesar de las diferencias en el comportamiento entre distintas estrategias de búsqueda, la estructura de los algoritmos para la búsqueda en entornos con incertidumbre se puede generalizar del siguiente modo 2. Las diferencias en el comportamiento se encuentran en la función de transición $F_T(s_u^{t-1}, c_u^t, \Delta T)$ y la función para seleccionar de la siguiente acción de control $F_S(s_u^{t-1}, \dots)$ y no en el procedimiento de búsqueda en sí.

La función de transición define cómo reaccionará el agente al aplicar una determinada acción de control c_u^t para pasar a un nuevo estado s_u^t en un cierto periodo de tiempo ΔT . El comportamiento durante esta transición depende en gran parte de la dinámica del UAV: para una misma acción, por ejemplo, cambiar de dirección para apuntar al norte, un dron de tipo quadcopter podrá activar los rotores de forma que se oriente en esta dirección manteniéndose estático en su posición mientras que un planeador realizará una maniobra de giro más amplia. Nuestra función de transición está más alineada con la dinámica de los drones quadcopters, que además de poder moverse en la dirección que indiquemos sin importar la orientación de este, pueden permanecer estáticos.

La función de selección de las acciones de control $F_S(s_u^{t-1}, \dots)$ calculará a partir del estado previo s_u^t , cuál debe ser la siguiente acción de control c_u^t , pudiendo tener en cuenta además del estado del agente, la creencia sobre el objetivo $b(v^t)$, factores del entorno como la dirección del viento o la forma del terreno, las posiciones del resto del enjambre o el historial de sus acciones de control previas hasta ese momento $c_u^{0:t-1}$. Esta función es la que marca el tipo de algoritmo de búsqueda que realizará el agente. A la hora de seleccionar los patrones de búsqueda, hemos usado patrones de barrido tradicionales y buscadores guiados por heurísticas. En concreto hemos implementado:

- Búsqueda de barrido de cortacesped, *LS* (Lawnmower Search).
- Búsqueda de barrido en espiral, *ESS* (Expanding Square Search).
- Búsqueda voraz guiándose por una heurística, *Greedy*.

De forma general para realizar la búsqueda del objetivo, el agente parte de un estado s_u^{t-1} y, tras aplicar la acción de control correspondiente, alcanza un nuevo estado s_u^t . El agente usa su sensor para observar el espacio de búsqueda, aplicando el algoritmo del RBF 1, actualizando la creencia sobre el objetivo y comprobando si se ha encontrado.

Algorithm 2 Pseudocódigo de los algoritmos de búsqueda

Require: $F_T(s_u^{t-1}, c_u^t, \Delta T), F_S(s_u^{t-1}, \dots)$ ▷ Transition and next-action functions

Require: $b(v^0), RBF(s_u^t, b(v^{t-1}))$ ▷ Initial belief, Recursive Bayesian Filter

Require: $s_{1:U}^0, T, \Delta T, U$ ▷ Initial state, search time, time interval, number of agents

```
1:  $N = T / \Delta T$  ▷ Number of time steps
2: for  $t = 1 : N$  &  $u = 1 : U$  do
3:    $c_u^t = F_S(s_u^{t-1}, \dots)$  ▷ Control action
4:    $s_u^t = F_T(s_u^{t-1}, c_u^t)$  ▷ Transition function
5:    $z_u^t, b(v^t) = RBF(s_u^t, b(v^{t-1}))$  ▷ Observatiton Step
6:   if  $z_u^t = D$  then ▷ Check if found
7:     return true ▷ Target found
8:   end if
9: end for
10: return false ▷ Target not found
```

5.3.1. Búsqueda de barrido de cortacesped

Este algoritmo de búsqueda de fuerza bruta cubre todo el espacio empezando en un extremo y moviéndose de lado a lado en largos tramos rectos dando dos giros de 90° al final de cada uno de ellos, hasta llegar al extremo opuesto a donde empezó, encontrar el objetivo o llegar al número máximo de pasos, según lo que ocurra antes. Antes de comenzar a hacer la búsqueda el agente se dirige hacia la esquina más cercana y desde ahí comenzará a buscar. Una pequeña optimización que se ha añadido es que el agente deje una separación con el borde del espacio de búsqueda igual al rango de detección del sensor, de esta forma se consigue que sea algo más eficiente por no tener que llegar hasta el borde y aún así hacer un barrido completo del espacio de búsqueda. Además, podemos controlar la forma del patrón de búsqueda ajustando la separación entre los tramos con el parámetro de configuración **separation**, definiendo si queremos una búsqueda más compacta o menos. Un ejemplo de este patrón se puede ver en la figura 5.2.

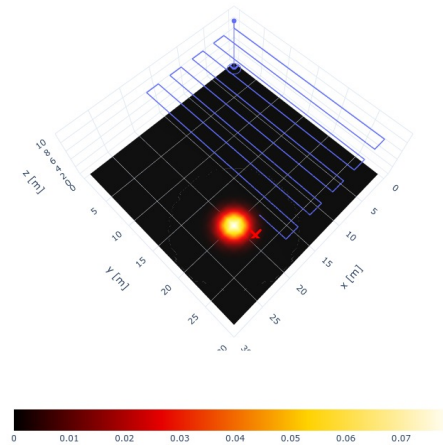


Fig. 5.2. Patrón de búsqueda de cortacesped

5.3.2. Búsqueda de barrido en espiral

Un recorrido en forma de espiral cuadrada que se va expandiendo desde el centro hacia los bordes. Al igual que en la búsqueda anterior, antes de realizar el patrón de búsqueda el agente debe posicionarse en un lugar específico, en este caso dirigiéndose al centro del mapa y desde allí empieza a trazar cuadrados de mayor y mayor tamaño. El algoritmo parará cuando toque alguno de los bordes, encuentre el objetivo o haya alcanzado el límite de pasos. Y de la misma forma que en la búsqueda de cortacesped, podemos ajustar la forma del patrón de búsqueda cambiando la separación entre tramos, haciendo que haga espirales más o menos grandes. Se puede ver una trayectoria de ejemplo en la figura 5.3.

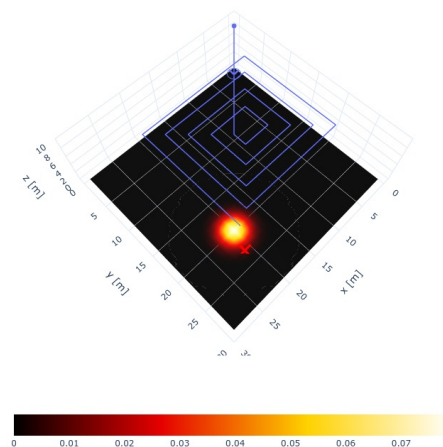


Fig. 5.3. Patrón de búsqueda de cuadrado creciente

5.3.3. Búsqueda voraz

Como ya hemos explicado en la sección 4.1, los algoritmos voraces o *greedy* se mueven guiados por una heurística, contemplando únicamente la mejor acción en el momento según la heurística, sin valorar si esa acción es la mejor a largo plazo, lo que puede llevar a que el agente se comporte de manera miópica. Hemos implementado dos heurísticas: una usando el criterio para la reducción de la miopía descrito anteriormente en 4.2 por el que el agente se ve guiado hacia el punto con la mayor probabilidad; y otra siendo una heurística local, escogiendo entre sus acciones disponibles la que más cambie la creencia. Comparando estas heurísticas vamos a poder apreciar los efectos que tiene la miopía sobre este tipo de estrategias y planificadores. En la figura 5.4 podemos ver la trayectoria del planificador voraz: el agente se mueve hacia la zona con mayor probabilidad guiándose por la heurística.

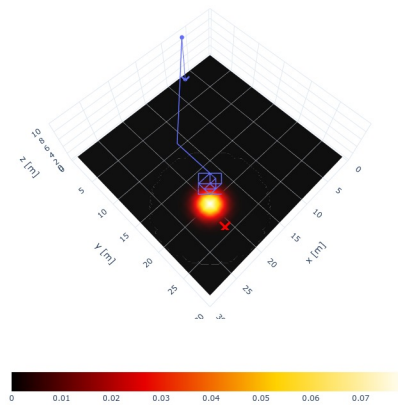


Fig. 5.4. Patrón de búsqueda de voraz

5.4. Resultados de las pruebas

Para poder analizar los resultados las búsquedas es necesario tener datos acerca de la operación de búsqueda. Para ello tras finalizar se graban los siguientes datos:

- Las posiciones iniciales y finales de los agentes y del objetivo.
- La distancia recorrida por cada agente y el objetivo, más concretamente la suma de las distancias a cada paso. En nuestro caso al ser nuestro objetivo estático la distancia siempre será 0.
- El número de pasos que han dado los agentes y el objetivo.
- Si se encontró al objetivo y qué agente lo hizo.

- La semilla usada para generar sus posiciones iniciales en caso de que se haya usado una posición aleatoria.

Además de cierta información para recrear la búsqueda: algoritmo usado para el movimiento de los agentes y nombre del archivo de configuración, que se codifica en el nombre del archivo en el que se guarda.

	Agent 0	Agent 1	Agent 2	Target
Initial position	[21.0 7.0]	[27.0 3.0]	[24.0 21.0]	[21 24]
Final position	[20.0 23.0]	[18.0 19.0]	[18.0 20.0]	[21 24]
Distance	313.848	361.127	277.279	0
Steps taken	27	27	27	0
Found Target	TRUE	FALSE	FALSE	0
Semilla	0	0	0	-

Fig. 5.5. Información obtenida en una búsqueda

Esta información se guarda en archivos CSV y con ella vamos a comparar la eficiencia y la eficacia de las distintas estrategias. Y gracias a que el procesamiento de estos archivos resulta sencillo, el análisis se puede paralelizar masivamente para ejecutarse con grandes cantidades de datos.

A partir de estos datos podemos evaluar el comportamiento de los algoritmos utilizando varias métricas de calidad: porcentaje de búsquedas exitosas o tasa de acierto, la media, la desviación típica y la distribución de los cuartiles sobre la distancia recorrida, y la comparación de la distancia (suma de todos los desplazamientos) y el recorrido de los agentes (diferencia entre la posición inicial y la final).

5.5. Interfaz gráfica

A partir de una primera versión en la que se podía mostrar el recorrido de un agente, se siguió mejorando para permitir varios agentes, diferenciar los distintos agentes asignándoles colores, mostrar de forma simplificada la visión del sensor...

La interfaz muestra el movimiento de los agentes en un espacio 3D mostrando los caminos que recorren y cómo se va actualizando distribución de la creencia en el espacio de búsqueda. Este escenario se crea usando la lista de posiciones de los agentes y el estado del mapa de probabilidad en cada instante para recrear una animación de la búsqueda.

Usando esta interfaz hemos podido arreglar errores encontrados durante el desarrollo, gracias a que podíamos ver exactamente qué es lo que estaban haciendo los agentes y, por ejemplo, ajustar los parámetros de las heurísticas usadas. Además de permitir exportar estas visualizaciones, por lo que también hemos podido usarla para generar todas las visualizaciones de esta memoria.

6. PROCESO DE EXPERIMENTACIÓN

Una vez hemos desarrollado el framework, se experimentará con diferentes escenarios de prueba sobre los que evaluar las estrategias de búsqueda. En esta sección se explican las pruebas realizadas y la motivación detrás de cada caso de prueba.

El proceso de experimentación se puede dividir en dos partes: el diseño y generación de las pruebas por una parte y la ejecución de esos casos de pruebas usando las distintas estrategias por otra.

6.1. Diseño de las pruebas

A la hora de establecer el caso de prueba se usa un archivo de configuración donde se instancia el problema definiendo los parámetros ya mencionados en 5.2.1. A los casos de prueba se les añadió un número máximo de pasos que podían dar para evitar que los planificadores voraces se pudieran quedar *atascados* de forma indefinida en un mínimo local debido al problema de la miopía comentado en 4.1 y que los algoritmos de búsqueda por fuerza bruta puedan recorrer todo el espacio de búsqueda. Este límite temporal se escogía de acuerdo al tamaño del mapa dando en principio suficiente tiempo para que todos los indicios fueran investigados o que los algoritmos de fuerza bruta puedan recorrer una parte suficientemente grande del espacio de búsqueda.

La posición de los indicios en cada prueba se ha fijado alejada del borde para evitar que, al elegir la posición de los agentes de forma aleatoria, algún agente se posicione justo encima del objetivo.

6.2. Tipos de pruebas

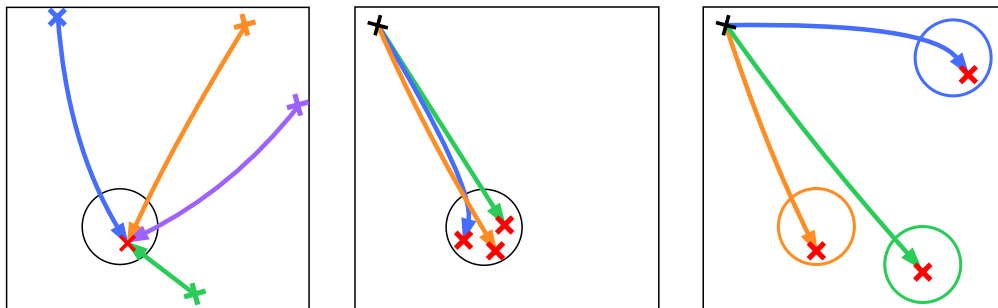
En concreto se han recreado 6 tipos de escenarios de pruebas, que explicaremos acompañados de visualizaciones esquemáticas para facilitar la comprensión de cada una de ellas 6.1 a 6.3.

En estas visualizaciones se representan en color negro elementos invariantes entre cada ejecución y usando colores más brillantes los que sí cambian entre ejecuciones. En cuanto a los símbolos usados: las cruces rojas indican la posición del objetivo y las cruces de otros colores indican la posición inicial de los agentes, las flechas indicarían el movimiento de los agentes durante las pruebas y los círculos los indicios de probabilidad.

Los siguientes escenarios se han planteado para comparar los algoritmos de búsqueda implementados en varias situaciones: 2 algoritmos de fuerza bruta y un algoritmo de guiado por heurística con 2 heurísticas a elegir. Además, en los casos con varios indicios,

la probabilidad inicial se puede distribuir de forma desigual entre ellos.

- Escenarios con un indicio fijo y un agente que aparece en el borde para buscar un objetivo cuya posición es fija. 6.1a
- Escenarios con un indicio fijo y un agente con una posición de inicio también fija, elegida de forma aleatoria cerca del borde del espacio de búsqueda, para buscar un objetivo cuya posición varía entorno al indicio de probabilidad. Estas están definidas en los archivos de pruebas acabados en *fixed*. 6.1b
- Escenarios con un agente fijo empezando en el borde del espacio y un indicio cuya posición se escoge de forma aleatoria en cada ejecución, siguiendo una distribución uniforme. El objetivo se coloca alrededor del indicio. Estas están definidas en los archivos de pruebas acabados en *random*. 6.1c
- Escenarios con varios indicios fijos y un agente que aparece en el borde para buscar un objetivo cuya posición es fija. 6.2a
- Escenarios con varios indicios fijos y un agente con una posición de inicio también fija, elegida de forma aleatoria cerca del borde del espacio de búsqueda, para buscar un objetivo cuya posición varía, apareciendo cerca de uno de los indicios. 6.2b



(a) Posición del indicio fija y agente aparece en el borde

(b) Posiciones del indicio y el agente fijas

(c) Posición del agente fija y el indicio varía

Fig. 6.1. Tipos de escenarios para comparar rendimiento entre algoritmos con un solo indicio.

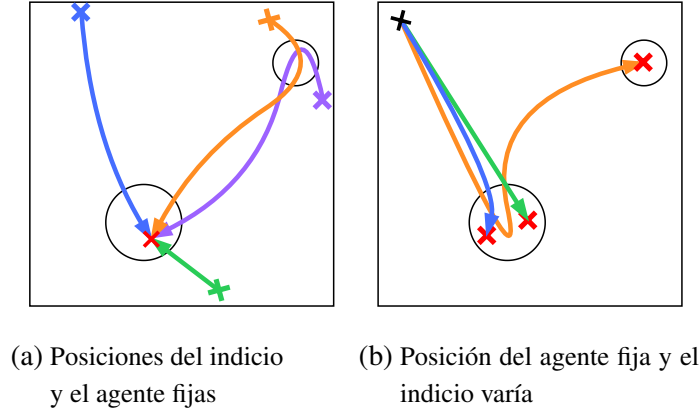


Fig. 6.2. Tipos de escenarios para comparar rendimiento entre algoritmos, con varios indicios.

Para la coordinación de múltiples agentes se hará un único tipo de prueba:

- Escenarios con un indicio fijo, cerca del centro del espacio, y un número de agentes que va aumentando, colocándose todos en una zona delimitada en el borde. 6.3

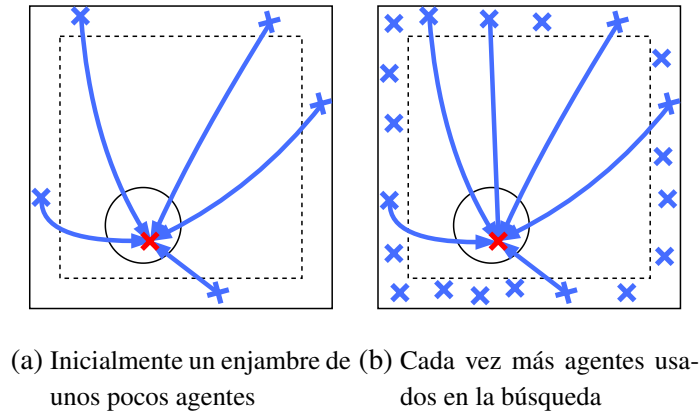


Fig. 6.3. Evolución del enjambre en la búsqueda multi-agente.

Todas estas pruebas, además, se han creado en 3 tamaños de escenario distintos: escenario *pequeños*: 30×30 , escenarios *medianos*: 50×50 y escenarios *grandes*: 100×100 . Obteniendo un total de 18 casos de prueba.

6.3. Ejecución de las pruebas

Una vez creadas las pruebas, debemos ejecutarlas. Para ello, partiendo de nuestro conjunto de casos de prueba, para cada caso de prueba se harán las ejecuciones un proceso parecido al ilustrado en 6.4: establecer la semilla del generador de números aleatorios (RNG), crear la situación inicial de la búsqueda $b(v^0)$, seleccionar el algoritmo de búsqueda configurándolo según los parámetros definidos en la prueba y realizar el proceso de búsqueda hasta que se encuentre el objetivo o se llegue al límite temporal establecido.

Una vez terminado el proceso de búsqueda, se guardarán los resultados y, si es posible, se mostrará una animación pudiendo visualizar el recorrido de los agentes, la posición del objetivo y los cambios de la creencia.

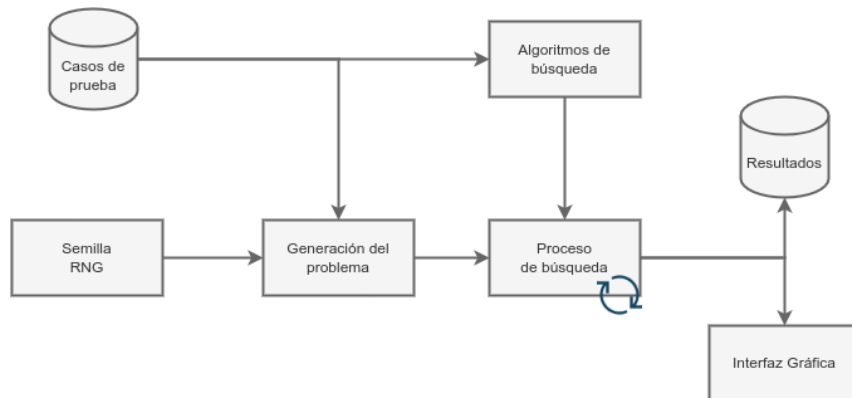


Fig. 6.4. Diagrama de la ejecución de un caso de prueba.

6.3.1. Entorno de ejecución de las pruebas

A la hora de ejecutar las pruebas se ha usado el entorno de ejecución de Python ofrecido por Google Colaboratory[21], usando el almacenamiento de Google Drive para guardar los archivos con los resultados generados en las distintas pruebas. Además de alojar los archivos de código en un repositorio de git[15] bajo una licencia MIT. Todos los archivos relacionados con el trabajo se encuentran en este repositorio, tanto las funciones como los archivos de configuración y los cuadernillos de Python usados para realizar las pruebas y el análisis de estas.

La ejecución de las pruebas se ha realizado en un cuadernillo de Python llamado `lanzar-pruebas.ipynb`. El cuadernillo se ha dividido en tres partes, una para cada uno de los experimentos: comparación de métodos de búsqueda para un solo indicio de probabilidad, comparación de métodos de búsqueda en la búsqueda multi-indicio y comprobación de la coordinación de múltiples agentes.

Las ejecuciones están comentadas y estructuradas de forma que se puedan entender haciendo referencia a esta memoria, y a las definiciones sobre los tipos de prueba 6.2.

7. ANÁLISIS DE RESULTADOS

Una vez explicadas las pruebas veamos los resultados de los experimentos. Como ya se ha mencionado, se han realizado 3 experimentos: el primero, comprobar la eficiencia y eficacia de algoritmos de búsqueda guiados por heurística frente a algoritmos de fuerza bruta; en el segundo, exponer a esos mismos algoritmos de búsqueda a un entorno con varios indicios de probabilidad; y en el tercero, observar la coordinación de los agentes en la búsqueda para distintos tamaños de enjambres.

Dentro de cada experimento se han realizado varios casos de prueba, de acuerdo a lo expuesto en la sección 6, obteniendo 4,000 puntos de datos para cada caso, guardados en archivos CSV. Todos estos datos se encuentran organizados en archivos comprimidos en el repositorio de código dentro del directorio `resultados`, correspondiéndole a cada prueba un archivo. Estos archivos se han nombrado siguiendo el formato:

<tamaño del mapa>-<número de indicios>-<número de agentes que participaran en la búsqueda>-<indicador sobre la posición de los indicios>.

Por ejemplo, el archivo *grande-indicios-1-agentes-1-fixed.zip*, contiene las pruebas para un caso de prueba en el que se usa 1 agente, para buscar en un espacio de búsqueda grande (100×100) con 1 indicio de probabilidad, siendo la posición del agente y del indicio fijas.

El análisis de los datos se ha hecho en varios cuadernos de Python, de nuevo, separando cada experimento en un cuaderno distinto. Estos cuadernos se han llamado `analisis-resultados.ipynb`, `analisis-resultados-multiindicio.ipynb` y `analisis-resultados-multiagente.ipynb`, y se han organizado de tal forma que todos casos de las pruebas sigan un orden parecido y sean fáciles de seguir en conjunto con esta memoria.

7.1. Experimento 1: Diferencia entre algoritmos de búsqueda por fuerza bruta y algoritmos guiados por heurísticas

En este primer experimento se evalúan diferentes estrategias de búsqueda. Los algoritmos de búsqueda que se eligieron son algoritmos que barren la totalidad del espacio de búsqueda, ya que un algoritmo como la búsqueda sectorial que se centra en un punto creando sectores triangulares que aproximan los arcos de un círculo, aunque funcionaría bien en un espacio de búsqueda con un solo indicio, en el caso de que hubiera más de un indicio dejaría zonas de interés sin cubrir o habría que introducir comportamiento adicional, por ejemplo, el movimiento para cambiar de un indicio a otro teniendo que también seleccionar el orden en el que se investigan, y dejaría de ser un algoritmo puramente de fuerza bruta. Por ello se usaron únicamente los patrones de cuadrado creciente y cortacesped.

Estos patrones de búsqueda tradicionales y de fuerza bruta se usarán para establecer una línea base con la que comparar a los algoritmos de búsqueda guiados por heurísticas.

7.1.1. Caso 1

Este es un caso de prueba de poca complejidad que sirve como un caso base sobre el que realizar los siguientes casos. El área de búsqueda es un cuadrado de 30 unidades de lado, un espacio pequeño para asegurar que sin importar la estrategia o la heurística usada todos los métodos tienen posibilidades de llegar al objetivo, teniendo que encontrarlo en menos de 90 pasos. En este área hay un indicio de probabilidad cerca del centro del mapa en la posición (20,10), con un objetivo estático situado en la posición (17,13). Para la búsqueda se ha usado 1 solo agente que, dependiendo de la semilla, cambiará su punto de inicio siempre en uno de los bordes del mapa. En la imagen 7.1 podemos ver una agregación de algunos casos de prueba, mostrando las posiciones en las que empiezan los agentes y la posición del objetivo y el indicio. Este caso de prueba se corresponde con el archivo de configuración `indicios-1-agentes-1.json`.

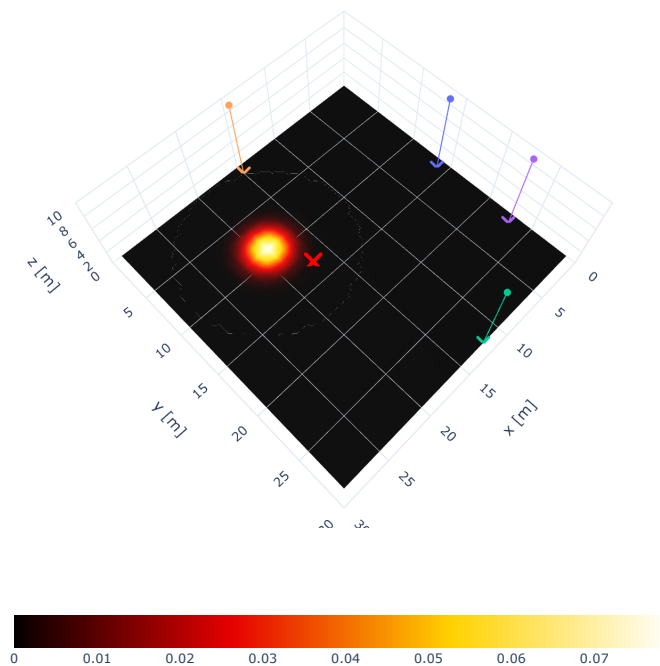


Fig. 7.1. Combinación de las situaciones iniciales del caso de prueba 1

Mirando el gráfico 7.2 podemos ver que la mayor parte de los intentos son exitosos para la mayoría las estrategias, aunque dado que el objetivo está cerca del centro del espacio de búsqueda, al usar el patrón de búsqueda de cortacesped la mayor parte de las

veces se queda corto y no llega a encontrar el objetivo.



Fig. 7.2. Porcentajes de intentos Exitosos para cada Estrategia

Ahora observando las estadísticas de la búsqueda de forma más detallada en forma de tabla 7.1, podemos ver que los dos planificadores voraces son más rápidos con una media de unos 23 pasos aunque menos consistentes que los buscadores por fuerza bruta y, dada la similitud de la distribución de sus estadísticas 7.3, podemos asumir que en escalas pequeñas se comportan de forma igual, el efecto de la miopía no se aprecia.

Strategy	mean	std	min	25 %	50 %	75 %	max
expanding	27.81	10.476	11.0	11.0	33.0	35.0	35.0
lawnmower	83.861	21.265	10.0	90.0	90.0	90.0	91.0
ldfs-heur	23.617	12.035	11.0	13.0	15.0	36.0	41.0
ldfs-myop	24.419	15.763	11.0	15.0	18.0	31.0	90.0

TABLA 7.1. ESTADÍSTICAS DE LA BÚSQUEDA

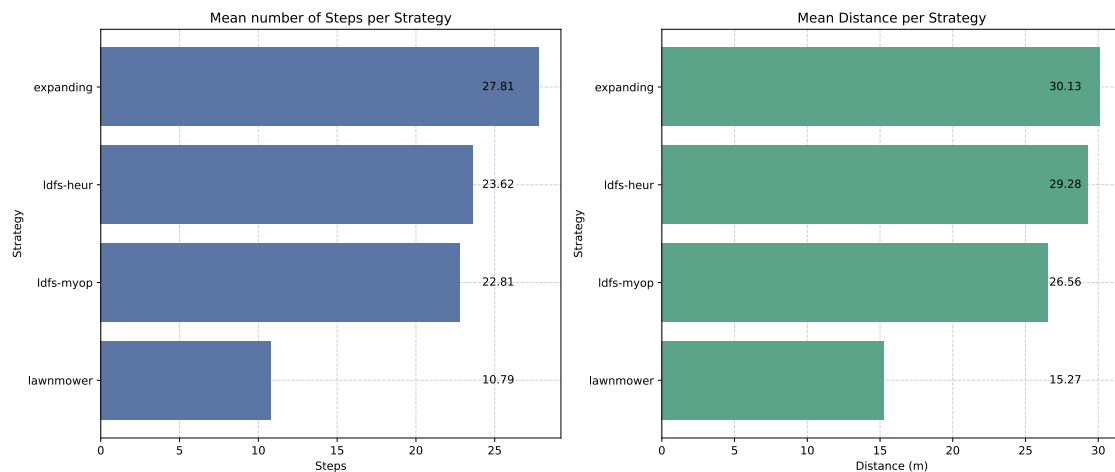


Fig. 7.3. Distribución de pasos y distancias de los casos exitosos

Cabe destacar que en la figura 7.3 solo se muestra la distribución de pruebas exitosas. La estrategia de lawnmower da la apariencia de ser más rápida porque los casos en los que es capaz de encontrar el objetivo son aquellos en que la posición inicial es cercana al objetivo.

7.1.2. Caso 2: Posición inicial del agente fija

Una vez visto este primer caso base, vamos a probar cómo de bien lo harán si la posición inicial del agente está fija. Para ello probaremos dos casos: un caso en el que el indicio también está fijo y un caso en el que la posición del indicio cambia esto se corresponde a los estilos de prueba 6.1b y 6.1c respectivamente.

De la misma forma que en el caso anterior, aquí presentamos una visualización compuesta agregando la posición de inicial de los agentes de algunas pruebas. En el resto de casos siguientes solo se incluirá si fuese necesario para comprender la prueba o los resultados obtenidos. Incluida al inicio de la sección para cada prueba en los cuadernillos de análisis en el repositorio de código del trabajo: `analisis-resultados.ipynb`, `analisis-resultados-multiagente.ipynb` y `analisis-resultados-multiagente.ipynb`, se encuentra una visualización de este estilo para cada una de las pruebas junto a todas las gráficas que no se han incluido para hacer este análisis, por si se quisieran visitar. Aunque en estas ilustraciones aparezcan varios indicios o varios objetivos, recalamos que son una superposición de las configuraciones iniciales de varios casos de prueba.

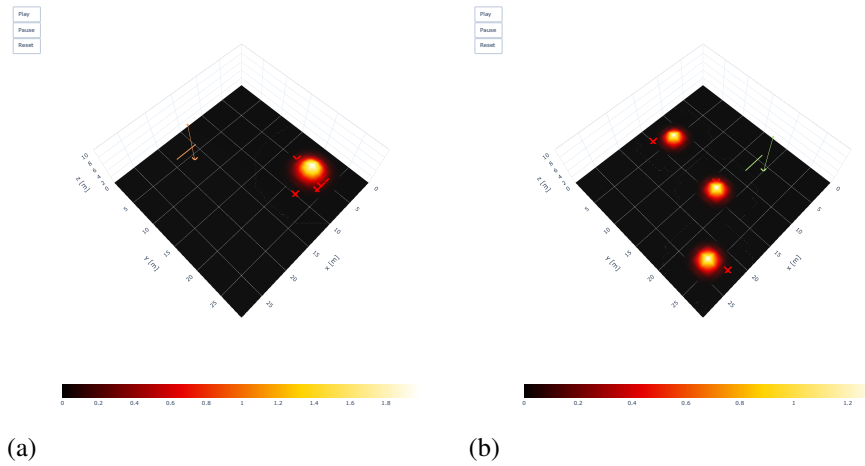


Fig. 7.4. Combinación de las condiciones iniciales de las pruebas: (a) prueba con indicios fijos, (b) prueba con indicios en posiciones aleatorias

Al mantener las posiciones de los agentes fijas, podemos comprobar cómo de capaces son los algoritmos a la hora de generalizar su comportamiento. Comparado con el caso inicial, en el que la posición desde la que empezaba era aleatoria, ahora el agente no puede aprovechar que en ciertas ocasiones aparecía algo más cerca del objetivo y por ello era capaz de encontrarlo. Primero analicemos el caso en que ambas posiciones están fijas. Esto nos permitirá ver cómo pequeñas variaciones afectan a un mismo trayecto.

Observando los porcentajes en los que se encuentra el objetivo para cada estrategia podemos ver que, ahora que el objetivo está más cerca del borde, el algoritmo de cuadrado creciente no es capaz de encontrarlo con tanta facilidad y usando el barrido de cortacesped sí que se encuentra en más casos, justo al contrario que en el caso anterior donde era el algoritmo de cortacesped el que tenía dificultades. Podemos ver que las diferencias entre usar una heurística miope y una en la que los efectos de la miopía están reducidos, siguen sin ser significativas ambos alcanzando el objetivo casi en el 100 % de las pruebas.

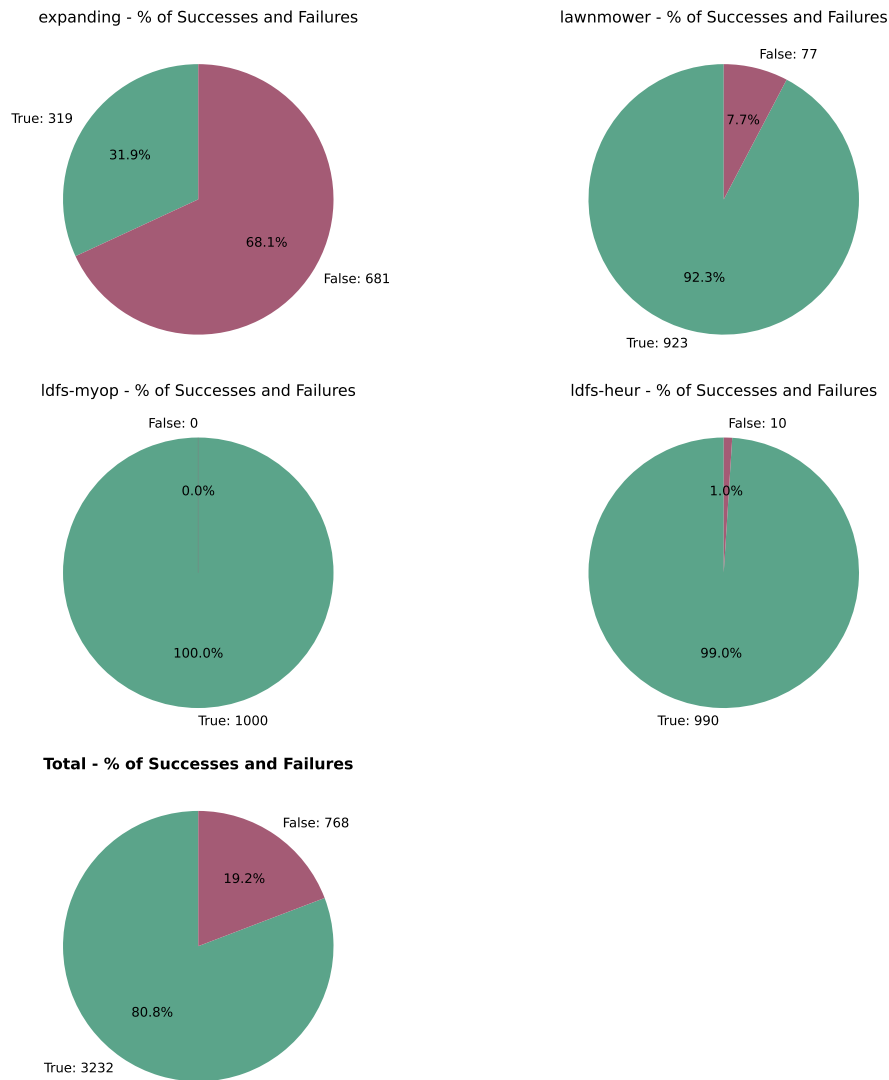


Fig. 7.5. Porcentaje de intentos exitosos por estrategia

Algo interesante que podemos ver en los histogramas de la distribución de los pasos 7.6 es que, al usar el algoritmo de fuerza bruta de barrido de cortacesped, hay 2 zonas distintas donde se concentra la probabilidad. Esto se debe a que la posición del objetivo va variando entre pruebas, pero siempre alrededor de un mismo indicio. El agente al hacer el barrido pasa cerca del indicio varias veces y estas dos acumulaciones de probabilidad equivaldrían a cada una de las pasadas, separadas aproximadamente por la distancia recorrida entre las pasadas, entendiendo que puede encontrarlo a la ida o a la vuelta. Más adelante, cuando probemos con espacio de búsqueda de tamaños mayores y con varios indicios, veremos estos mismos tipos de distribución.

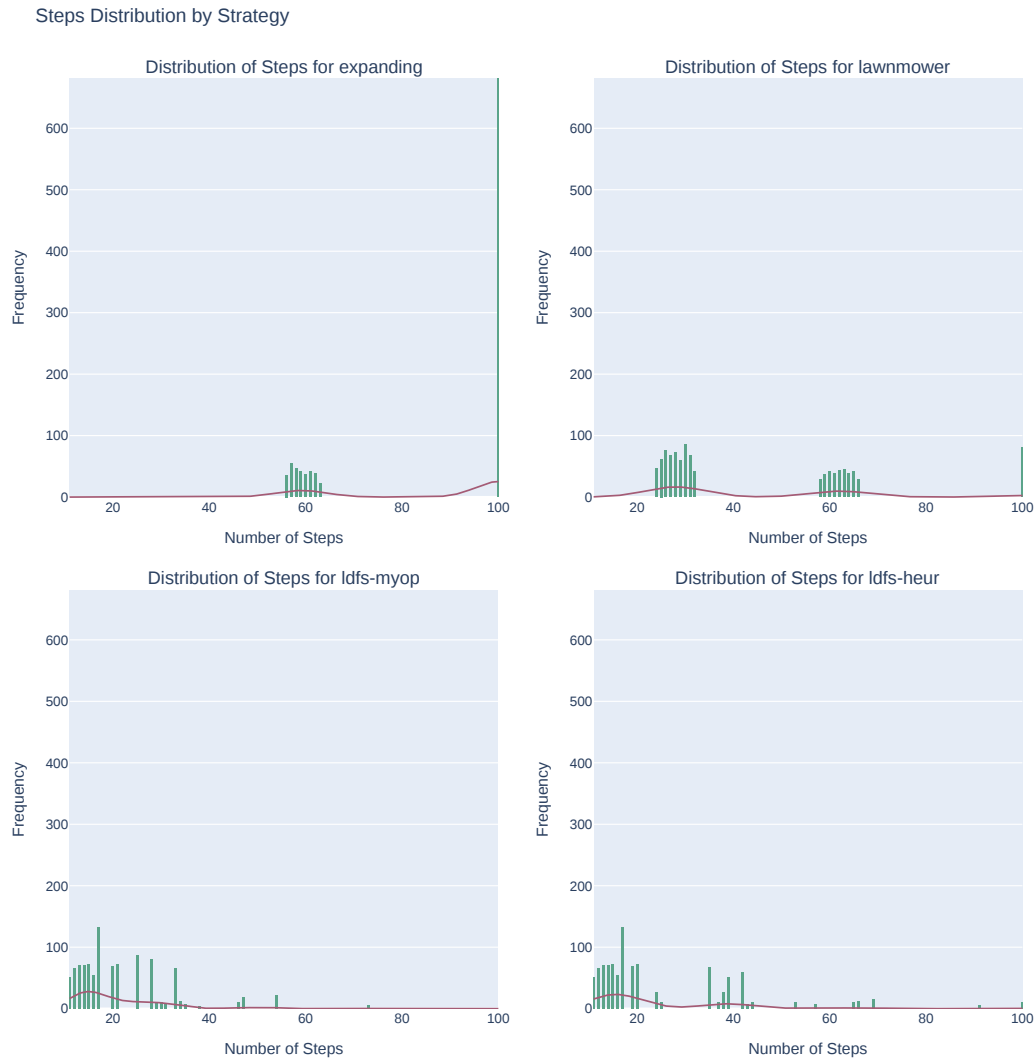


Fig. 7.6. Distribución de los pasos tomados para cada estrategia - Posiciones fijas

Ahora cuando probamos a posicionar el inicio de forma aleatoria obtenemos unos resultados bastante diferentes. La posición del inicio se ha seleccionado siguiendo una distribución uniforme para cada coordenada, excluyendo del rango posiciones muy cercanas al borde dejando un pequeño margen, para evitar la posibilidad, aunque poco probable, de que el agente se encuentre encima del objetivo, de la misma forma en la que la posición inicial de los agentes está limitada a un margen alrededor del borde.

Podemos ver ahora que en una parte importante de los intentos no se consigue encontrar el objetivo. Tienen gran dificultad en especial las estrategias por fuerza bruta que, al no poder barrer el espacio de búsqueda en su totalidad, dependen de que el objetivo esté dentro del rango que pueden explorar dentro del límite temporal. Igual que en la prueba anterior, al buscar apoyándose en heurísticas se obtienen resultados mejores. Ya en esta prueba podemos ver el efecto del factor de reducción de la miopía, habiendo una fracción importante, casi un 25 %, de los intentos en que el planificador miope no fue capaz

de encontrar el objetivo. En estos casos, el objetivo apareció demasiado lejos para que detectase la distribución de probabilidad.



Fig. 7.7. Porcentaje de intentos exitosos para cada estrategia - Posición del indicio aleatoria

En la siguiente figura 7.8 podemos apreciar más detalles de la búsqueda y comprender algo mejor cómo funcionan los distintos métodos. La gráfica muestra la función de distribución acumulada (CDF) de la cantidad de pasos hasta llegar al objetivo. Lo primero que podemos observar es que en ambos casos las estrategias de fuerza bruta siguen una distribución uniforme. Esto se puede entender por el siguiente razonamiento: ambas estrategias, dada una cantidad suficientemente grande de tiempo, son capaces de recorrer el espacio de búsqueda al completo sin solapamiento.

por lo que se puede hacer un mapeo 1 a 1 entre todas las posiciones del espacio y una lista ordenada con los movimientos del agente. Así a cada punto del espacio le corresponde un único índice en la lista, el momento en el que el agente pasará por esa posición. La posición del objetivo se elige de forma aleatoria siguiendo una distribución uniforme,

lo que sería equivalente a que se tome un índice de forma aleatoria en la lista siguiendo otra distribución uniforme. La probabilidad de que acabe encontrándolo corresponde a la probabilidad de que ese índice sea menor a la cantidad de pasos que puede dar el agente, nos quedaría por tanto una distribución uniforme.

Ahora para el planificador voraz podemos ver una distribución más parecida a una distribución normal. Esto se puede justificar de la siguiente manera: gracias a la heurística se recorre el espacio de búsqueda de forma más estratégica y podemos asumir que, en la mayor parte de los casos, el agente irá directo hacia el indicio encontrando el objetivo poco después, por lo que lo que determina la distancia recorrida no es más que la distancia entre el objetivo y el agente, o sea, la distancia entre un punto fijo y uno aleatorio en el espacio, que es un evento aleatorio e independiente entre pruebas. De acuerdo al Teorema del Límite Central, la suma de suficientes eventos aleatorios independientes, la distribución que resulta es una distribución normal. Algo podemos ver claramente es cuando empieza a afectar la miopía. A partir de los 20 pasos aparece una larga cola, el planificador deja de seguir la distribución normal. Debido al efecto de la miopía, llega un momento que el agente se mueve de forma aleatoria porque ha “agotado” la probabilidad a su alrededor y ya no tiene información para guiarse. Ahora el agente se parece más a los buscadores de fuerza bruta que recorren el espacio sin información del objetivo y con cierta probabilidad de encontrarlo en su camino.



Fig. 7.8. Función de distribución de los intentos para cada estrategia - Posición del indicio aleatoria.

Ahora tomando los datos de la búsqueda podemos superponerlos con los de una gráfica de una distribución normal y obtener que la distribución de los pasos se parece razonablemente a una distribución normal 7.9. Para el resto de estrategias, se puede ver en la gráfica conjunta 7.10 la forma aproximada de las distribuciones y ver que: los planificadores voraces siguen la distribución normal, aunque la gráfica queda distorsionada por ese 25 % de casos sin éxito del planificador miope y la búsqueda por fuerza bruta es una línea horizontal que igual se ve influenciada por la cantidad de intentos fallidos en el extremo derecho.

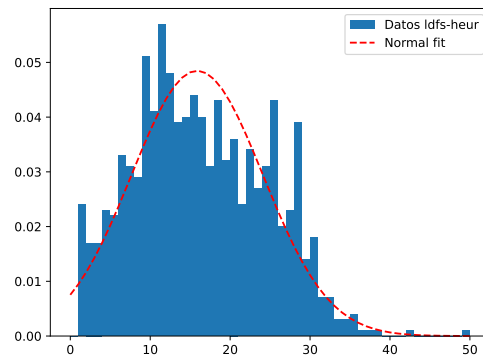


Fig. 7.9. Distribución normal ajustada, superpuesta sobre la distribución de pasos para el planificador con la heurística de reducción de miopía.

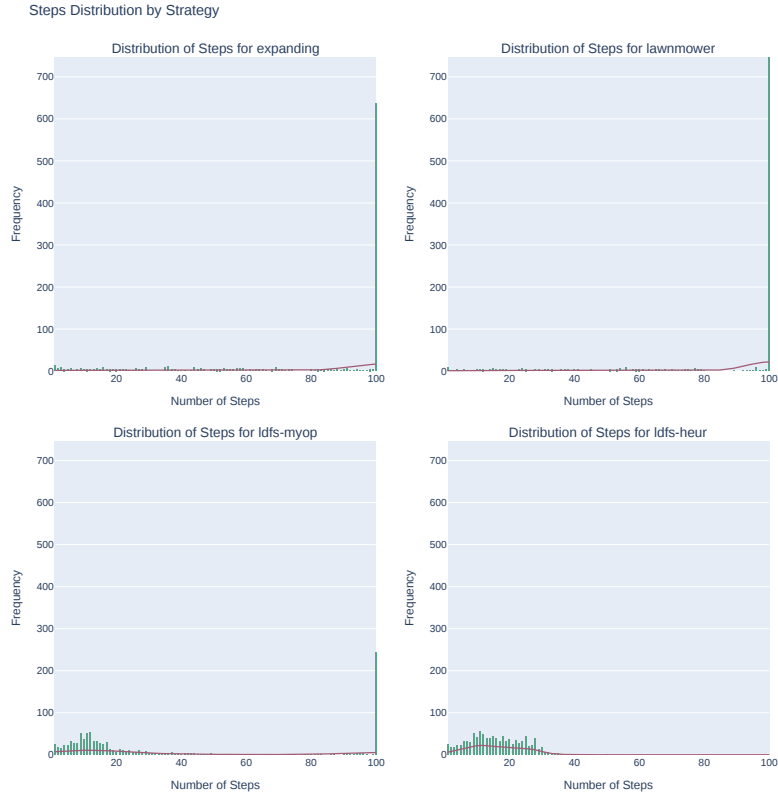


Fig. 7.10. Distribución de los pasos tomados por cada estrategia - Posición del indicio aleatoria.

7.1.3. Caso 3: Ampliar el tamaño del área de búsqueda

Este tercer caso es similar al primero, con la diferencia de que ahora el espacio de búsqueda es mayor, ampliándose a 50×50 y, posteriormente, a 100×100 . Este caso de prueba está definido en los archivos de configuración `mediano-indicios-1-agentes-1.json` y `grande-indicios-1-agentes-1.json`.

Dado que estos mapas son más grandes, a los agentes se les ha dado más tiempo para encontrar el objetivo, aumentando el límite de la búsqueda a de 100 a 200 pasos para el caso de 50×50 y llegando a 1200 en los casos más grandes con un espacio de 100×100 . Además de este cambio se les ha dado a los algoritmos de fuerza bruta una separación mayor entre los tramos, para que puedan recorrer más espacio. Tras la ejecución de las pruebas, la distribución de los pasos se puede ver en la figura 7.11, observamos varios patrones interesantes que desglosaremos uno a uno.

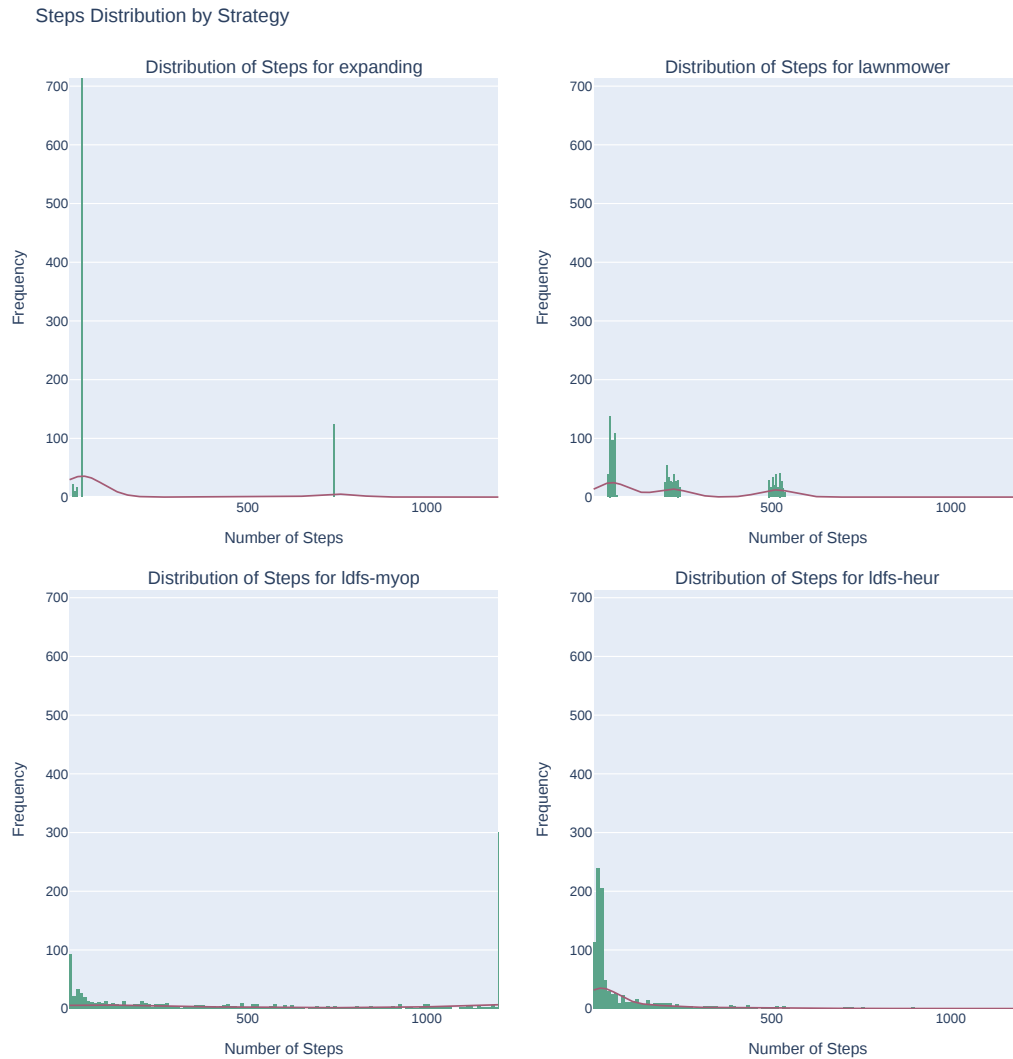


Fig. 7.11. Distribución de los pasos tomados por cada estrategia - Escenario grande.

Primero tener en cuenta la configuración inicial de la prueba 7.12: el indicio está bastante centrado aunque también relativamente cerca del borde, por lo que en todas las estrategias hay un pico al inicio, estos son los casos en los que el agente ha aparecido bastante cerca de esa esquina y ha sido capaz de encontrarlo antes.

Podemos ver un gran pico en la búsqueda de cuadrado creciente, esto se puede explicar viendo que hay un segundo pico menor por lo que ya que el patrón que dibuja es muy similar entre pruebas, siempre encuentra el objetivo en la misma cantidad de pasos. Algo parecido pasa con el patrón de cortacesped, que como ya hemos mencionado, esas dos zonas de probabilidad se corresponde a cuando el agente hace un barrido de ida y otro de vuelta, habiendo más variedad por el tiempo que tarde en posicionarse en la esquina apropiada que también marca que lo encuentre antes, dependiendo de si empieza yendo de izquierda a derecha o de derecha a izquierda.

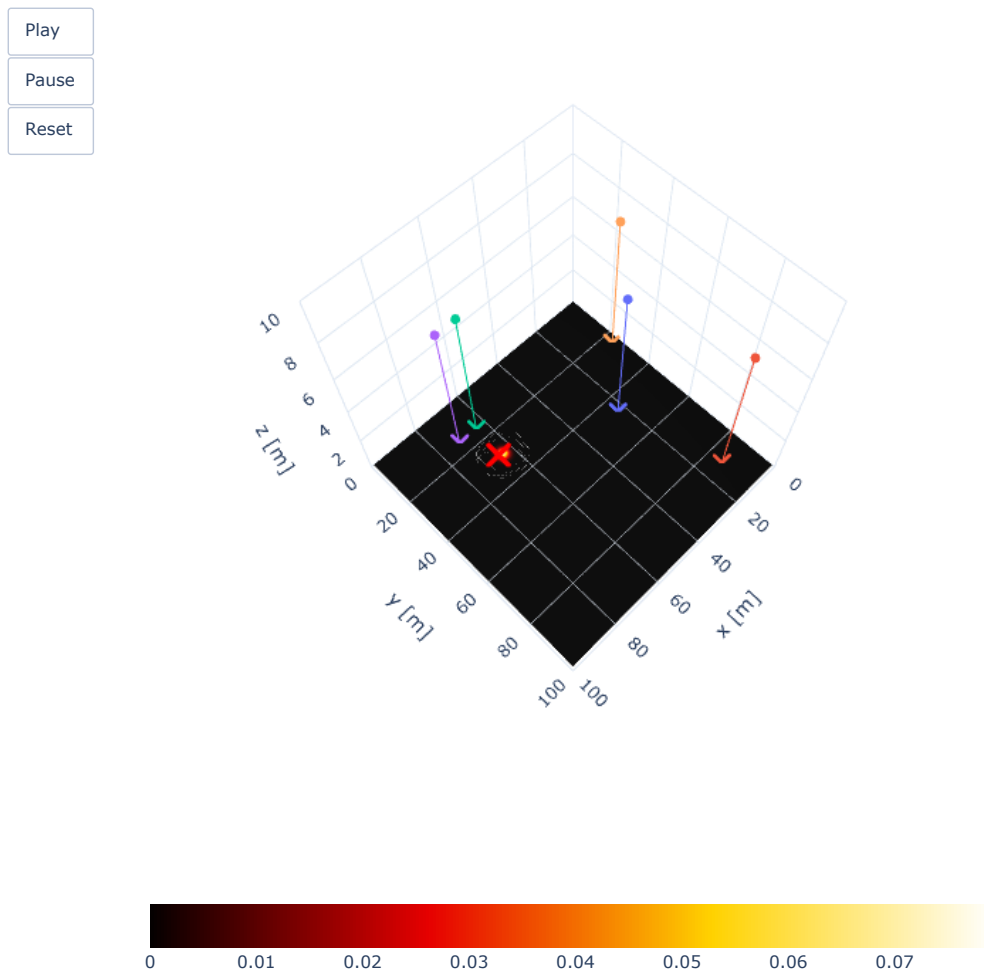


Fig. 7.12. Combinación de las situaciones iniciales del - Caso 3: Escenario grande.

Algo que se ve en parte reflejado en la tasa de éxito de cada estrategia 7.13 es la importancia del punto de inicio. El patrón de cortacesped con el límite temporal que hemos puesto puede explorar en torno a la mitad del espacio de búsqueda. Entonces, aproximadamente la mitad de las ocasiones empieza en la mitad del espacio *equivocada* y antes de llegar al objetivo debe parar por el límite temporal.



Fig. 7.13. Porcentaje de intentos exitosos por estrategia - Caso 3: escenario grande.

7.1.4. Caso 4: Ampliar el tamaño del área de búsqueda manteniendo la posición del agente fija

Por último, veamos qué ocurre cuando se mantiene la posición del agente fija en un espacio más grande. Al igual que en el tercer caso, se probaron con tamaños de escenario mediano y grande además de probar a dejar la posición del indicio fija y hacerla variar siguiendo una distribución uniforme.

Al igual que en el caso 2, como no poseen la ventaja que les da empezar en posiciones aleatorias, ahora necesitan muchos más pasos de media para encontrar el objetivo.

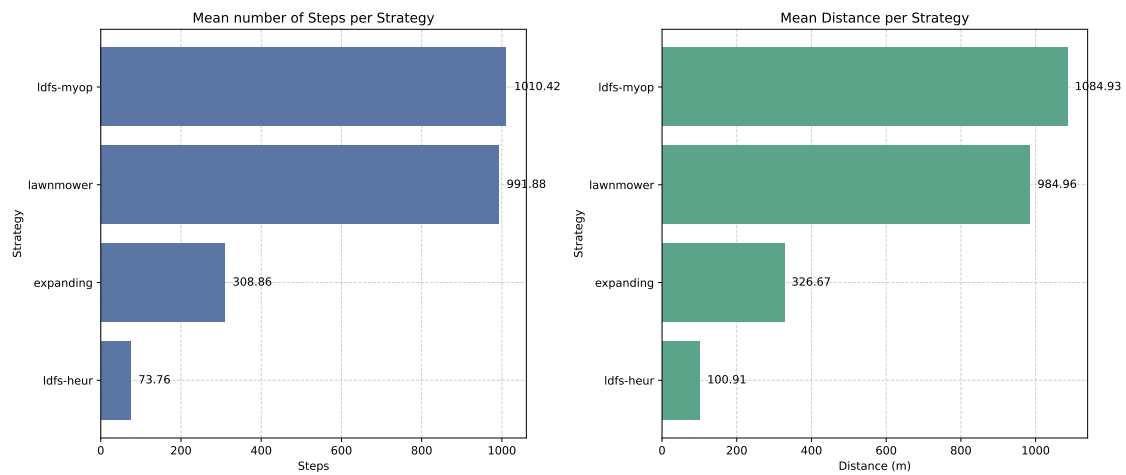


Fig. 7.14. Distancia media recorrida por cada estrategia - Caso 4

Algo que podemos ver fácilmente es que el algoritmo voraz con la heurística de reducción de miopía gracias a que usa la información global de la creencia es capaz de generalizar su comportamiento a estos mapas de mayor tamaño, yendo siempre hacia el indicio, el punto de mayor probabilidad. No solo eso, sino que también lo hace de forma extremadamente rápida. Mientras que usar la fuerza bruta se vuelve algo menos efectiva llegando al objetivo cuanto más aumenta el tamaño. Aunque esto se compensa con el hecho de que ahora tienen más tiempo para realizar la búsqueda, podemos ver cómo encontrar el objetivo es una cuestión de suerte en el histograma. Y para el agente con una heurística local el efecto de la miopía cobra gran relevancia. El agente se queda atrapado por el efecto de esta y no es capaz de llegar al objetivo y ni siquiera reaccionar a la distribución de probabilidad en la mayor parte de los casos. Esto es debido a que la probabilidad es igual en la mayor parte del mapa, lo que le hace recorrer un camino aleatorio o *random walk*.

7.2. Experimento 2: Búsqueda multi-indicio

En este segundo experimento quisimos ver si las mismas conclusiones que hemos sacado en el anterior experimento se mantienen cuando hay más de un indicio de búsqueda. Tras haber visto unos resultados en los que las heurísticas, y en especial la heurística con corrección de miopía, eran bastante eficientes a la hora de encontrar el objetivo y navegar el espacio de búsqueda, sorprende ver que han desempeñado tan pobremente.



Fig. 7.15. Porcentaje de pruebas exitosas por estrategia - Experimento 2: Búsqueda multindicio

El factor de corrección guía al agente hacia el punto con mayor probabilidad. Esto funcionaba muy bien cuando tratábamos únicamente con un indicio, pero ahora que tenemos varios no es tan efectivo. Se produce un efecto similar a la *oscilación del gradiente* cuando se entrena una red de neuronas artificial. Si aplicamos una tasa de aprendizaje demasiado alta, el modelo realiza una sobrecompensación y toma pasos demasiado grandes por lo que no es capaz de converger y termina oscilando de un lado a otro del gradiente. Ambos casos se pueden solucionar intentando pasar a un modelo que también incorpore información más local, reduciendo el learning rate o creando una heurística que combine el comportamiento global con el local.

Podemos verlo en la trayectoria del agente 7.16. En vez de seleccionar uno de los indicios, explorarlo completamente y continuar al otro; oscila entre ellos nunca llegando a explorar ninguno.

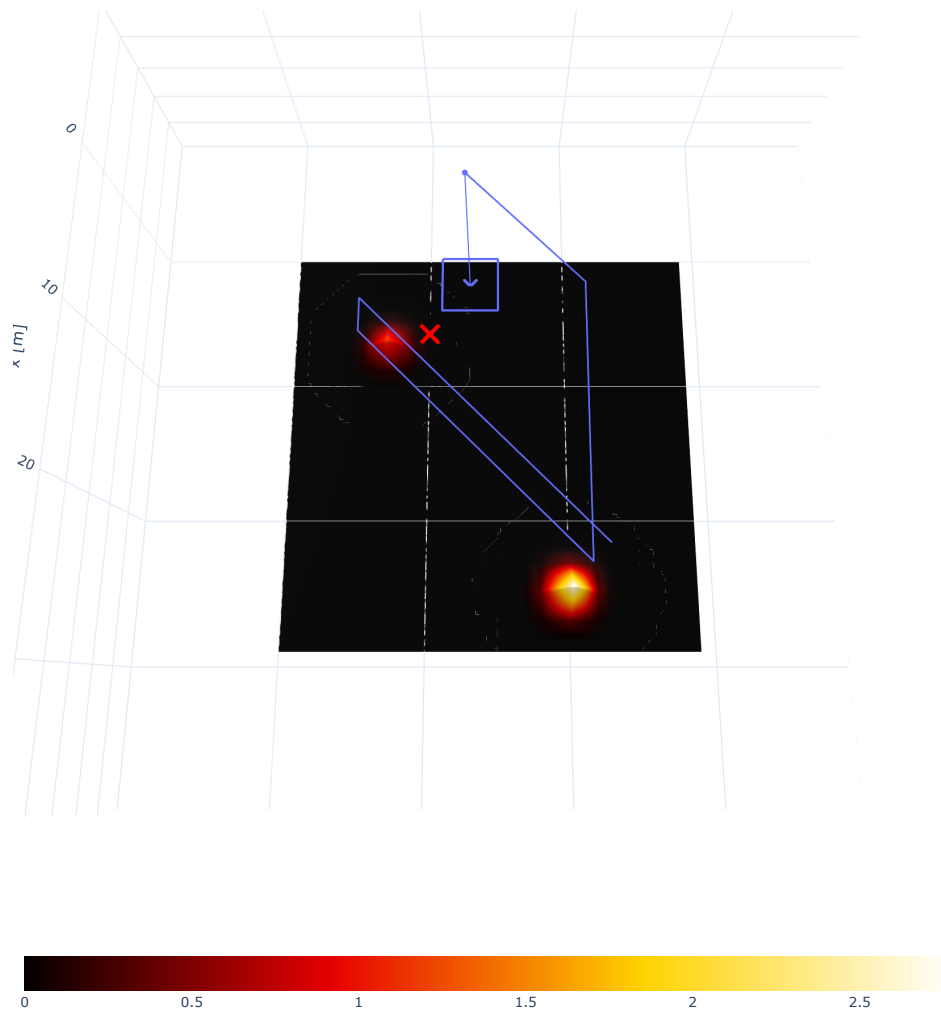


Fig. 7.16. Agente guiado por la heurística global, rebotando entre los indicios

Los modelos por fuerza bruta obtienen aproximadamente los mismos resultados, el tamaño del mapa y sobre todo la posición del objetivo son los factores que más condicionan la efectividad de estos.

7.3. Experimento 3: Búsqueda multiagente

Como hemos visto en las pruebas anteriores, las estrategias de búsqueda que usan heurísticas son más eficientes a la hora de encontrar los objetivos, al guiarse por la distribución de probabilidad. Aunque presentan también problemas que heurísticas más completas podrían solucionar.

Ahora vamos a comprobar cómo de eficientes son si aumentamos el número de agentes con los que podemos buscar. Para estas pruebas en los distintos escenarios no estableceremos un número fijo de agentes, sino que este irá aumentando. Así para cada uno de

los casos de prueba, se irá aumentando la cantidad de agentes que participan en la búsqueda, dejando el resto de la situación idéntica: tamaño del espacio de búsqueda, posición del objetivo, distribución inicial de probabilidad...

En este experimento solo se realizarán pruebas de un tipo como se indicó anteriormente 6.2. Estas pruebas consistirán en un escenario de tamaño: pequeño, mediano y grande; con un indicio cerca del centro del mapa y el objetivo colocado de forma aleatoria dentro de él. Los agentes se colocan alrededor del borde de forma aleatoria también, y se irá aumentando el tamaño del enjambre, considerando tamaños de 1, 10, 50 y 100 agentes.

La posición de los agentes se elige de forma aleatoria teniendo en cuenta dos restricciones:

- **Los agentes se colocan en el borde del área de búsqueda:** se toma una distancia de separación desde el borde alrededor de toda la zona de búsqueda, y los agentes solo se podrán colocar en esta zona. De esta forma podemos simular cómo desplegaríamos nuestro enjambre desde uno (o varios) de los bordes del área de búsqueda en una situación real.

Este borde se ha intentado que se mantenga entre un cuarto y un tercio del espacio de búsqueda en cada lado. Así creemos que se obtiene un balance: se deja a los agentes un área lo suficientemente grande para realizar la búsqueda, pero también tenemos suficiente espacio para generar las posiciones iniciales de estos.

- **Los agentes deben permanecer a una distancia mínima:** como ya hemos mencionado, uno de los parámetros de la búsqueda es la distancia mínima que debe mantenerse entre los agentes. Durante la búsqueda los movimientos que llevarían a que la distancia mínima no se respetase son infinitamente penalizados, por lo que los agentes son capaces de evitar que se llegue a una configuración *ilegal*. Partiendo de un estado válido en la que un movimiento en cualquier dirección llevase a que se incumpla la separación mínima entre agentes, el agente puede permanecer quieto y la restricción se cumpliría al permanecer en un estado válido.

Sin embargo, durante el desarrollo de las pruebas hemos observado que, si se parte desde una de estas configuraciones ilegales, el comportamiento de los agentes puede ser errático y en muchas ocasiones no consiguen solucionar el conflicto; si se da una situación en la que ninguno de sus posibles movimientos, incluido permanecer en el sitio, permite mantener esta distancia, todos los movimientos serán penalizados infinitamente y contendrán la misma información de acuerdo la heurística: el agente escogerá uno al azar describiendo un camino aleatorio y muy posiblemente seguirá violando la restricción.

Una solución sería aumentar las capacidades del agente, pasando de un planificador voraz a uno en la que la ventana de decisión sea mayor. La solución que implementamos en cambio, consiste en evitar que se puedan producir estas situaciones directamente. Los agentes se colocarían de forma que haya tanto espacio entre ellos

como dicte la distancia mínima. Para esto se van colocando los agentes uno a uno de forma iterativa y probando para cada uno muchas posibles posiciones, en concreto, 10,000 – un número arbitrariamente grande que nos permite asegurar con cierta confianza la posibilidad de conciliar la posición del agente actual con la del resto. En el caso de que no se consigan colocarse, la prueba falla y no se ejecuta, y posteriormente analizamos posibilidades: aumentar el área donde se pueden seleccionar posiciones, disminuir la distancia entre los agentes... Esto se hace con el raciocinio de que si la configuración actual no es válida para el objetivo de la prueba, es necesario realizar un mejor diseño de esta. Es esta restricción, la que ha hecho que no intentemos probar con tamaños de enjambres mayores, los agentes simplemente no se podían desplegar sin que unos pisaran a otros.

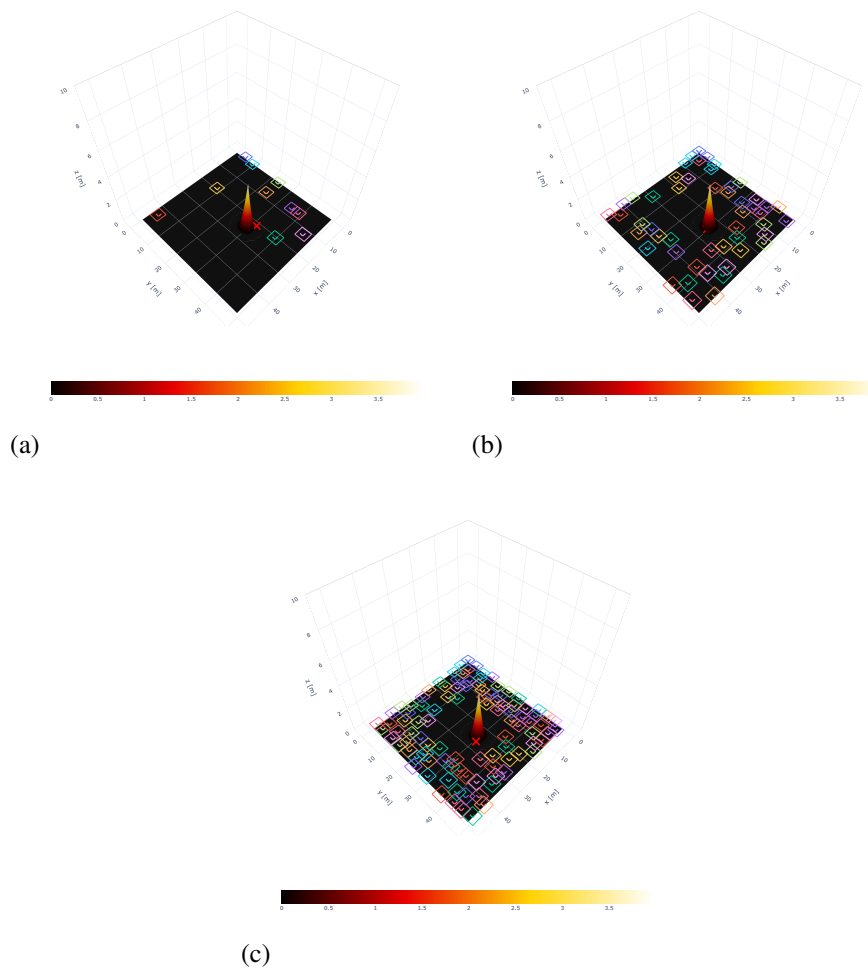


Fig. 7.17. Progresión del tamaño del enjambre para el escenario de búsqueda mediano:
(a) Diez agentes, (b) Cincuenta agentes, (c) Cien agentes

Cabe decir que en el primer caso de prueba, cuando el espacio donde se realiza la búsqueda es más pequeño, los tamaños de enjambre que se han estudiado son 1, 10 y 50. No se han usado más agentes porque el tamaño es bastante reducido y apenas cabían los 50 agentes. En los otros casos de prueba sí que se han probado hasta 100 agentes.

Ahora podemos ver cómo, en general, los casos de prueba para todos los tamaños de escenario y de enjambre los agentes son extremadamente exitosos y eficientes a la hora de encontrar el objetivo.

Espacio	Swam size	% aciertos	mean	min	25 %	50 %	75 %	max
Pequeño	1	100.0	30.54	1.0	20.0	29.0	45.0	51.0
	10	100.0	7.32	1.0	2.0	5.0	10.0	64.0
	50	100.0	1.412	1.0	1.0	1.0	1.0	13.0
Mediano	1	100.0	19.575	3.0	15.0	20.0	24.0	50.0
	10	100.0	11.002	2.0	9.0	11.0	13.0	24.0
	50	100.0	7.574	2.0	6.0	8.0	9.0	16.0
	100	100.0	6.751	2.0	5.0	7.0	8.0	16.0
Grande	1	99.6	87.033	14.0	33.75	41.0	53.0	1200.0
	10	100.0	25.104	10.0	21.0	25.0	30.0	61.0
	50	100.0	19.484	10.0	16.0	19.0	23.0	46.0
	100	100.0	18.139	10.0	15.0	17.0	21.0	44.0

TABLA 7.2. RESULTADOS POR TAMAÑO DE ENJAMBRE Y ESPACIO DE BÚSQUEDA: PORCENTAJE DE ACIERTOS, MEDIA Y SEPARACIÓN EN CUARTILES DE PASOS HASTA EL OBJETIVO.

En la tabla podemos ver cómo cuantos más agentes hay, más rápido se encuentra el objetivo, lo que se puede entender pensando en el área donde se colocan los agentes al inicio. Cuanto más cerca esté del borde interior, más fácil le será llegar objetivo, que está en el centro del mapa.

Algo que esperábamos ver es que al aumentar la cantidad de agentes, los movimientos de estos estarían restringidos por los otros agentes. Observaríamos una discrepancia entre la distancia recorrida, la suma de todos los movimientos a lo largo de la trayectoria; y el desplazamiento, la diferencia entre la posición inicial y final. Todos los agentes reportarían haberse movido una cierta cantidad de pasos, pero casi no se habrían separado de su posición de inicio. En la figura 7.18, podemos ver que el desplazamiento sí es menor que la distancia, pero la diferencia no es tan apreciable o significativa:

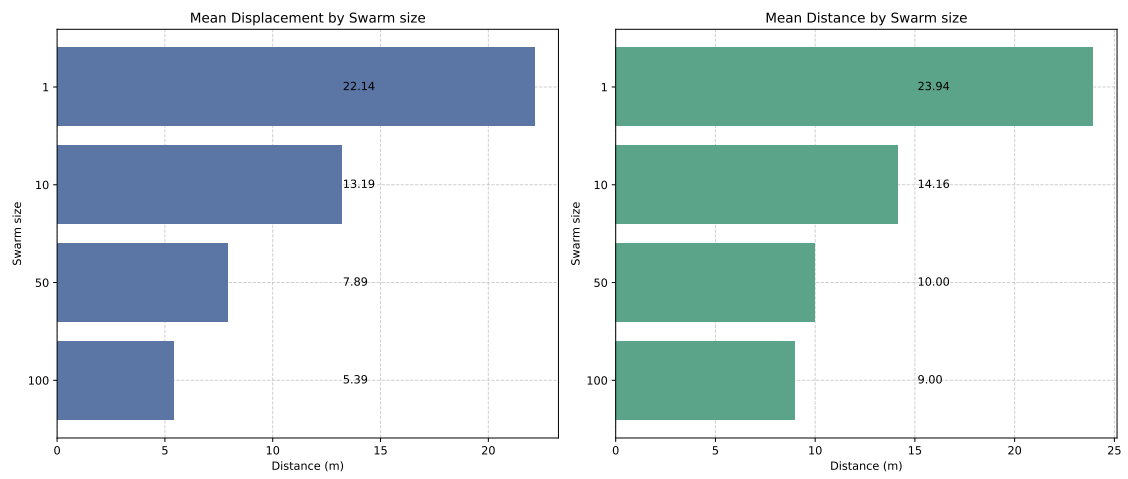


Fig. 7.18. Desplazamiento y distancia recorrida por tamaño de enjambre - Escenario mediano

8. CONCLUSIONES

Tras el análisis de los datos podemos ver que los algoritmos de búsqueda de fuerza bruta pierden eficacia a medida que aumenta la superficie de búsqueda ya que, al no incorporar ninguna información del mapa de creencia, deben recorrer la región por completo. Además, podemos apreciar el efecto de la miopía en los agentes basados en algoritmos voraces, haciendo que no pueda encontrar el objetivo en la mayor parte de los espacios más grandes, donde la probabilidad de encontrar mínimos locales es mayor.

En cuanto al uso de múltiples agentes, no hemos encontrado un punto a partir del cual añadir otro agente provoque un aumento en el tiempo requerido para encontrar el objetivo. Esto se puede deber al tamaño de la región a explorar y la restricción que mantiene alejados a los UAVs, lo que permite que se diversifiquen y exploren el espacio de forma más eficiente. Además, al acotar el número de agentes que pueden participar en la búsqueda para no entrar en colisión en la primera iteración, se permite que todos los agentes puedan moverse inicialmente.

8.1. Trabajo futuro

Una posible línea de trabajo sería comprobar si estos resultados y conclusiones se mantienen para la búsqueda de objetivos móviles. El modelo de simulación ya permite simular el comportamiento de un objetivo en movimiento que se mueva usando una matriz con las probabilidades de transición a las celdas vecinas, actualizando también la distribución de la creencia en el área búsqueda 3.3. El modelo se podría refinar para recrear comportamientos más sofisticados y esto también podría incitar ciertas modificaciones a los algoritmos de fuerza bruta, haciendo que se adapten al patrón de movimiento del objetivo de la misma forma que, por ejemplo, el patrón de búsqueda en sectores se altera al aplicarlo en entornos marítimos para tener en cuenta la deriva debida a la marea[22].

Además, como hemos visto, aunque el criterio de corrección de la miopía ha permitido que el agente navegue en espacios grandes sin quedar atrapado en mínimos locales, en cuanto aparecen varios indicios de probabilidad, el agente presenta un problema de oscilación entre los indicios, por lo que es necesario desarrollar nuevas y mejores heurísticas, además de incluir un mayor horizonte de planificación para las decisiones de los agentes, permitiéndoles valorar el resultado de una cadena de acciones más larga y dejando atrás los modelos de búsqueda voraces por modelos más sofisticados.

Y por último, este framework de simulación tiene ciertas limitaciones y a pesar del diseño inicial que se planteó intentando hacerlo lo más flexible posible, se han hecho modificaciones para poder incorporar las necesidades de otros proyectos. Por lo que un rediseño podría permitir extender el framework para crear un software más robusto e in-

corporar ciertas optimizaciones en el rendimiento para permitir visualizaciones en tiempo real.

BIBLIOGRAFÍA

- [1] B. Mayo, *Los drones de los zapadores paracaidistas sobrevuelan la zona devastada por la dana en busca de desaparecidos* - EFE, es-ES, nov. de 2024. [En línea]. Disponible en: <https://efe.com/espana/2024-11-06/drones-zapadores-paracaidistas-dana/> (Acceso: 01-06-2025).
- [2] United Nations General Assembly. ‘Transformar nuestro mundo: la Agenda 2030 para el Desarrollo Sostenible.’ (2015), [En línea]. Disponible en: <https://www.ods.cr/es/17-objetivos-de-desarrollo-sostenible>.
- [3] Comisión Europea, *Reglamento de Ejecución (UE) 2021/664 de la Comisión de 22 de abril de 2021 sobre un marco regulador para el U-Space (Texto pertinente a efectos del EEE)*, Diario Oficial de la Unión Europea, L 139, p. 161-183, abr. de 2021. [En línea]. Disponible en: https://eur-lex.europa.eu/eli/reg_impl/2021/664/oj.
- [4] P. Lanillos, ‘Minimum time search of moving targets in uncertain environments,’ Tesis doct., Universidad Complutense de Madrid, jul. de 2013.
- [5] P. Lanillos, E. Besada, G. Pajares, J. Ruz, IEEE y R. Japan, ‘Minimum Time Search for Lost Targets using Cross Entropy Optimization,’ *2012 Ieee/rsj International Conference on Intelligent Robots and Systems (Iros)*, pp. 602-609, oct. de 2012. doi: [10.1109/IROS.2012.6385510](https://doi.org/10.1109/IROS.2012.6385510).
- [6] S. Perez-Carabaza, E. Besada-Portas, J. A. Lopez-Orozco y J. M. de la Cruz, ‘A Real World Multi-UAV Evolutionary Planner for Minimum Time Target Detection,’ en *Proceedings of the Genetic and Evolutionary Computation Conference 2016*, ép. GECCO ’16, New York, NY, USA: Association for Computing Machinery, jul. de 2016, pp. 981-988. doi: [10.1145/2908812.2908876](https://doi.org/10.1145/2908812.2908876). [En línea]. Disponible en: <https://doi.org/10.1145/2908812.2908876> (Acceso: 04-08-2024).
- [7] J. N. Eagle, ‘The Optimal Search for a Moving Target When the Search Path is Constrained,’ *Operations Research*, vol. 32, n.º 5, pp. 1107-1115, 1984, Publisher: INFORMS. [En línea]. Disponible en: <https://www.jstor.org/stable/170656> (Acceso: 05-09-2024).
- [8] S. Carabaza, E. Besada, J. Lopez-Orozco y J. de la Cruz, ‘Planificador de Búsqueda en Tiempo Mínimo en un Sistema de Control de RPAS,’ ene. de 2016. doi: [10.17979/spudc.9788497498081.0785](https://doi.org/10.17979/spudc.9788497498081.0785).
- [9] S. Pérez Carabaza, E. Besada Portas, J. A. López Orozco y G. Blanco Rodríguez, ‘Modelado y optimización de misiones de búsqueda de objetivos mediante UAVs,’ spa, en *XL Jornadas de Automática: libro de actas. Ferrol, 4-6 de septiembre de 2019*, 2019, ISBN 978-84-9749-716-9, págs. 574-581, Section: XL Jornadas de

- Automática: libro de actas. Ferrol, 4-6 de septiembre de 2019, Servizo de Publicacións, 2019, pp. 574-581. [En línea]. Disponible en: <https://dialnet.unirioja.es/servlet/articulo?codigo=7031368> (Acceso: 02-05-2024).
- [10] A. Papoulis y S. Pillai, *Probability, Random Variables, and Stochastic Processes* (McGraw-Hill electrical and electronic engineering series). McGraw-Hill, 2002. [En línea]. Disponible en: <https://books.google.es/books?id=YYwQAQAAIAAJ>.
 - [11] L. Stone, ‘Theory of Optimal Search,’ en *Mathematics in Science and Engineering*, 1975.
 - [12] T. Furukawa, F. Bourgault, B. Lavis y H. Durrant-Whyte, ‘Recursive Bayesian search-and-tracking using coordinated uavs for lost targets,’ en *Proceedings 2006 IEEE International Conference on Robotics and Automation, 2006. ICRA 2006.*, ISSN: 1050-4729, mayo de 2006, pp. 2521-2526. doi: [10.1109/ROBOT.2006.1642081](https://doi.org/10.1109/ROBOT.2006.1642081). [En línea]. Disponible en: <https://ieeexplore.ieee.org/document/1642081/?arnumber=1642081> (Acceso: 02-09-2024).
 - [13] S. Pérez Carabaza, ‘Multi-UAS minimum time search in dynamic and uncertain environments,’ es, <http://purl.org/dc/dcmitype/Text>, Universidad Complutense de Madrid, 2019. [En línea]. Disponible en: <https://dialnet.unirioja.es/servlet/tesis?codigo=248422> (Acceso: 02-05-2024).
 - [14] P. L. Pradas, ‘Minimum Time Search of Mobile Targets in Uncertain Environments,’ 2013.
 - [15] Y. Brotón Gutiérrez, J. P. Llerena Caña y J. García Herrero, *Drone Lab | Grupo GIAA del Dpto. Informática*, ver. 1.0.0. [En línea]. Disponible en: https://giaa.uc3m.es/giaa_drone_lab.
 - [16] C. R. Harris et al., ‘Array programming with NumPy,’ *Nature*, vol. 585, n.º 7825, pp. 357-362, sep. de 2020. doi: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2). [En línea]. Disponible en: <https://doi.org/10.1038/s41586-020-2649-2>.
 - [17] W. McKinney, ‘Data Structures for Statistical Computing in Python,’ en *Proceedings of the 9th Python in Science Conference*, S. van der Walt y J. Millman, eds., 2010, pp. 56-61. doi: [10.25080/Majora-92bf1922-00a](https://doi.org/10.25080/Majora-92bf1922-00a).
 - [18] P. T. Inc. ‘Collaborative data science.’ (2015), [En línea]. Disponible en: <https://plot.ly>.
 - [19] P. S. Foundation, *csv — CSV File Reading and Writing*, <https://docs.python.org/3/library/csv.html>, 2020. (Acceso: 09-01-2025).
 - [20] P. S. Foundation, *json - JSON encoder and decoder*, <https://docs.python.org/3/library/json.html>, 2021. (Acceso: 09-01-2025).
 - [21] Google Research, *Google Colaboratory*, <https://colab.research.google.com>, 2024. (Acceso: 09-01-2025).

- [22] SmarterEveryDay, *Why This Zig-Zag Coast Guard Search Pattern is Actually Genius - Smarter Every Day 268*, ene. de 2022. [En línea]. Disponible en: <https://www.youtube.com/watch?v=aoXJfuPaFF8> (Acceso: 02-09-2024).